STUDENT KIT | EDITION 1.0

Web Development Student Kit

From complete beginner to job-ready full stack developer

Transforming Students Through Real-World Learning and Internships.

Compiled November 2026. Tool interfaces and prices change often; always check official sites.

Table of Contents

Table of Contents 2

Welcome to the SKYROVIX Student Kit 12

How to use this handbook . . . 12

Kit map: the 20 modules . . . 12

1. What is Web Development? 15

Types of web work . . . 15

Why learn it? . . . 15

2. How Websites Work 15

The journey of one page request . . . 16

3. Frontend, Backend and Full Stack 17

4. HTML Fundamentals 17

Your first web page . . . 17

Essential elements . . . 18

5. CSS Fundamentals 19

The box model . . . 19

Layout with Flexbox and Grid . . . 19

6. JavaScript Fundamentals 20

Making a page react to the user (the DOM) . . . 21

Asynchronous JavaScript and fetching data . . . 21

7. React.js Introduction 22

8. Node.js Introduction 23

9. APIs and Databases 23

10. Git and GitHub in 5 Minutes 24

11. The Full Stack Roadmap 25

Capstone project: Student Management App . . . 25

12. Chapter Assignments and Self-Check 25

Assignments . . . 25

Self-check questions . . . 26

Official learning resources . . . 26

End of Part 01 . . . 26

HTML Complete Guide **28**

2.1 HTML Introduction . . . 28
2.2 HTML Document Structure . . . 28
2.3 Elements and Attributes . . . 28
2.4 Headings and Paragraphs . . . 29
2.5 Links and Images . . . 29
2.6 Lists and Tables . . . 29
2.7 Forms and Input Elements . . . 30
2.8 Semantic HTML . . . 31
2.9 HTML5 Features . . . 31
2.10 SEO-Friendly HTML . . . 31
2.11 Accessibility . . . 32
2.12 Practical Projects . . . 32
Screenshots: what your pages look like . . . 32
Module 2: Common Mistakes and Troubleshooting . . . 33
Module 2: Best Practices and Security . . . 33
Module 2: Exercises . . . 33
Module 2: Interview Questions . . . 34
Module 2: Official Documentation and Resources . . . 34

CSS Complete Guide **36**

3.1 CSS Introduction . . . 36
3.2 Syntax and Selectors . . . 36
3.3 Colours and Backgrounds . . . 36
3.4 Typography . . . 36
3.5 Box Model . . . 37
3.6 Margin and Padding . . . 37
3.7 Flexbox . . . 37
3.8 CSS Grid . . . 38
3.9 Positioning . . . 38
3.10 Transitions and Animations . . . 38
3.11 Responsive Web Design . . . 39
3.12 Media Queries . . . 39
3.13 CSS Variables . . . 39

3.14 Practical Projects . . . 39
Screenshots: responsive layout in action . . . 40
Module 3: Common Mistakes and Troubleshooting . . . 42
Module 3: Best Practices and Security . . . 42
Module 3: Exercises . . . 42
Module 3: Interview Questions . . . 43
Module 3: Official Documentation and Resources . . . 43

JavaScript Complete Guide 45

4.1 JavaScript Introduction . . . 45
4.2 Variables and Data Types . . . 45
4.3 Operators . . . 45
4.4 Conditions and Loops . . . 46
4.5 Functions . . . 46
4.6 Arrays and Objects . . . 46
4.7 DOM Manipulation . . . 47
4.8 Events . . . 47
4.9 ES6+ Features . . . 47
4.10 Promises and Async/Await . . . 48
4.11 Fetch API . . . 48
4.12 Error Handling . . . 48
4.13 Local Storage . . . 49
4.14 Practical Projects . . . 49
Screenshot: the finished to-do app . . . 49
Module 4: Common Mistakes and Troubleshooting . . . 50
Module 4: Best Practices and Security . . . 50
Module 4: Exercises . . . 50
Module 4: Interview Questions . . . 51
Module 4: Official Documentation and Resources . . . 51

Frontend Development 53

5.1 Frontend Architecture . . . 53
5.2 React.js Complete Guide . . . 53
5.3 JSX . . . 53
5.4 Components and Props . . . 54

5.5 State and Hooks . . . 54
Forms in React . . . 55
5.6 React Router . . . 56
5.7 Context API . . . 57
5.8 API Integration . . . 57
5.9 TypeScript Fundamentals . . . 58
5.10 Next.js Introduction . . . 59
5.11 Next.js Routing . . . 59
5.12 Server and Client Components . . . 60
5.13 Tailwind CSS . . . 61
5.14 Frontend Projects . . . 61
Module 5: Common Mistakes and Troubleshooting . . . 61
Module 5: Best Practices and Security . . . 62
Module 5: Exercises . . . 62
Module 5: Interview Questions . . . 63
Module 5: Official Documentation and Resources . . . 63

Backend Development 65

6.1 Backend Development Fundamentals . . . 65
6.2 Node.js Complete Guide . . . 65
6.3 npm and Packages . . . 65
6.4 Express.js . . . 66
6.5 REST API Development . . . 66
6.6 HTTP Methods and Status Codes . . . 67
6.7 Middleware . . . 68
6.8 Authentication . . . 69
6.9 Authorization . . . 69
6.10 JWT . . . 69
6.11 Password Hashing . . . 70
6.12 File Uploads . . . 71
6.13 Error Handling . . . 71
6.14 Backend Projects . . . 72
Module 6: Common Mistakes and Troubleshooting . . . 72
Module 6: Best Practices and Security . . . 72

Module 6: Exercises . . . 72
Module 6: Interview Questions . . . 73
Module 6: Official Documentation and Resources . . . 73

Database Management 75

7.1 Database Fundamentals . . . 75
7.2 SQL Fundamentals . . . 75
7.3 MySQL . . . 76
7.4 PostgreSQL . . . 77
7.5 Database Design . . . 77
7.6 Tables and Relationships . . . 77
7.7 Primary and Foreign Keys . . . 78
7.8 CRUD Operations . . . 78
7.9 Joins and Queries . . . 79
7.10 Supabase . . . 80
7.11 Firebase . . . 80
7.12 Database Authentication . . . 80
7.13 Database Security . . . 80
7.14 Practical Database Projects . . . 81
Module 7: Common Mistakes and Troubleshooting . . . 81
Module 7: Best Practices and Security . . . 81
Module 7: Exercises . . . 81
Module 7: Interview Questions . . . 82
Module 7: Official Documentation and Resources . . . 82

AI Tools for Web Developers 84

ChatGPT . . . 84
Claude . . . 85
Google Gemini . . . 85
Google Antigravity . . . 86
Cursor . . . 86
GitHub Copilot . . . 87
Lovable . . . 87
Bolt.new . . . 88
v0 . . . 88

Replit AI . . . 89
Comparison table . . . 89
How to write good prompts . . . 89

Visual Studio Code Complete Guide 92

Install . . . 92
Interface tour . . . 92
Useful extensions . . . 92
HTML, CSS and JavaScript workflow . . . 92
Create a React project . . . 93
Git and GitHub in VS Code . . . 93
Debugging . . . 93
Keyboard shortcuts (Windows/Linux; use Cmd for Ctrl on Mac) . . . 93

Google Antigravity Complete Guide 95

Install and first setup . . . 95
Understanding agents . . . 95
Build a website with an agent . . . 95
Writing effective prompts . . . 95
Best practices . . . 96

Cursor AI Complete Guide 98

Install and setup . . . 98
Main features . . . 98
Workflows . . . 98

Figma UI/UX Design Guide 101

Getting started . . . 101
Core concepts . . . 101
Design workflow . . . 101
Design system basics . . . 101

Supabase Complete Guide 104

Create a project . . . 104
Tables and relationships . . . 104
Authentication . . . 104
Row Level Security (RLS) . . . 105
Storage . . . 105

Firebase Complete Guide 107

Setup . . . 107
Core services . . . 107
React integration . . . 107
Security rules . . . 107
Hosting . . . 108
Supabase vs Firebase . . . 108

Domain Name Registration Guide 110

Domain extensions . . . 110
Choosing a good name . . . 110
Registration steps (general) . . . 110
GoDaddy and BigRock . . . 110
DNS explained . . . 111
Connecting a domain to hosting . . . 111
Common errors . . . 111

Vercel Deployment Complete Guide 113

Step 1: Prepare the project . . . 113
Step 2: Push to GitHub . . . 113
Step 3: Import into Vercel . . . 113
Environment variables . . . 113
React vs Next.js . . . 114
Custom domain . . . 114
Updating the site . . . 114
Troubleshooting . . . 114
Final conclusion . . . 114

Advanced Full Stack Development 116

17.1 Full Stack Architecture . . . 116
17.2 Frontend and Backend Integration . . . 116
17.3 Authentication Systems . . . 117
17.4 Role-Based Access Control . . . 118
17.5 Payment Gateway Integration . . . 118
17.6 E-commerce Development . . . 118
17.7 Admin Dashboard Development . . . 119

17.8 Real-Time Applications . . . 120
17.9 Web Security . . . 120
17.10 Performance Optimization . . . 121
17.11 SEO . . . 121
17.12 Web Application Testing . . . 122
17.13 Progressive Web Apps . . . 122
17.14 Production Best Practices . . . 123
Module 17: Common Mistakes and Troubleshooting . . . 123
Module 17: Best Practices and Security . . . 124
Module 17: Exercises . . . 124
Module 17: Interview Questions . . . 125
Module 17: Official Documentation and Resources . . . 125

Practical Real-World Projects 127

1. Personal Portfolio Website . . . 127
2. Business Website . . . 128
3. Landing Page . . . 128
4. Task Management Application . . . 129
5. Blog Application . . . 130
6. Student Management System . . . 130
7. E-commerce Website . . . 131
8. Admin Dashboard . . . 131
9. Course Management Platform . . . 132
10. AI-Powered Web Application . . . 133
11. Complete Full Stack Application . . . 133
Module 18: Common Mistakes and Troubleshooting . . . 134
Module 18: Best Practices and Security . . . 134
Module 18: Exercises . . . 134
Module 18: Interview Questions . . . 135
Module 18: Official Documentation and Resources . . . 135

Professional Developer Skills 137

19.1 GitHub Portfolio Development . . . 137
19.2 Technical Documentation . . . 137
19.3 Code Review . . . 138

19.4 Debugging Techniques . . . 138
19.5 Team Collaboration . . . 138
19.6 Project Management . . . 139
19.7 Freelancing Fundamentals . . . 139
19.8 Client Communication . . . 139
19.9 Resume Preparation . . . 140
19.10 LinkedIn Profile Development . . . 140
19.11 Technical Interview Preparation . . . 140
19.12 Career Roadmap . . . 141
Module 19: Common Mistakes and Troubleshooting . . . 141
Module 19: Best Practices and Security . . . 141
Module 19: Exercises . . . 142
Module 19: Interview Questions . . . 142
Module 19: Official Documentation and Resources . . . 142

Final Capstone Project: SKYROVIX Student Management System 144

20.1 Project Planning . . . 144
20.2 Requirement Analysis . . . 144
20.3 UI/UX Design . . . 145
20.4 Frontend Development . . . 145
20.5 Backend Development . . . 145
20.6 Database Integration . . . 146
20.7 Authentication and Security . . . 146
20.8 API Integration . . . 146
20.9 Testing and Debugging . . . 146
20.10 GitHub Repository Management . . . 147
20.11 Domain Configuration . . . 147
20.12 Production Deployment . . . 147
20.13 Final Project Documentation . . . 147
Module 20: Common Mistakes and Troubleshooting . . . 147
Module 20: Best Practices and Security . . . 148
Module 20: Exercises . . . 148
Module 20: Interview Questions . . . 148
Module 20: Official Documentation and Resources . . . 148

Glossary **150**

Conclusion and Next Steps **150**

Welcome to the SKYROVIX Student Kit

This handbook teaches you web development step by step. It assumes you are a complete beginner, yet every chapter also has an **Advanced Corner** for learners who want to go deeper. You can study alone: every concept has an example, an exercise and a list of common mistakes.

How to use this handbook

- Read a chapter, then **type the code yourself**. Copy-pasting teaches very little.
- Do every exercise before moving on. Struggling for 10 minutes is part of learning.
- Keep a notes file of errors you met and how you fixed them.
- Look for the red **Common Mistake** boxes. They save hours.

Kit map: the 20 modules

Module	Topic	Status
01	Web Development Fundamentals	Written
02	HTML Complete Guide	Written
03	CSS Complete Guide	Written
04	JavaScript Complete Guide	Written
05	Frontend Development (React, Next.js, Tailwind)	Written
06	Backend Development	Written
07	Database Management (SQL, MySQL, PostgreSQL)	Written, with real query output
08	AI Tools for Web Developers	Written (overview level)
09	Development Tools (VS Code)	Partly written (VS Code only)
10	Google Antigravity	Written (overview level)
11	Cursor AI	Written (overview level)
12	Figma UI/UX Design	Written (overview level)
13	Supabase	Written (overview level)
14	Firebase	Written (overview level)
15	Domain Registration and DNS	Written (overview level)
16	Hosting and Deployment	Written (Vercel only)
17	Advanced Full Stack Development	Written
18	Real-World Projects	Written
19	Professional Developer Skills	Written
20	Final Capstone Project	Written

A note on accuracy
AI tools, pricing and screens change every few months. For those parts, this kit will state the date of verification and link to official pages. Where a price or screen could not be verified, it will say so instead of guessing.

MODULE 01

Introduction to Web Development

Websites, how they work, and the complete full stack roadmap

1. What is Web Development?

Web development is the work of building websites and web applications: the things you open in a browser such as an online shop, a college portal or a video site. A **website** mainly shows information. A **web application** lets you do things (log in, order food, submit an assignment).

Types of web work

Type	What it means	Example
Static website	Same content for every visitor	Personal portfolio, company brochure
Dynamic website	Content changes based on data or user	News site, blog with comments
Web application	Interactive software in the browser	Google Docs, Zomato, online exam portal
E-commerce	Products, cart and payments	Online store

Why learn it?

- Every business needs a web presence, so demand is steady.
- You can start with only a laptop and free tools.
- Freelancing, remote work and building your own products are all possible.
- It teaches problem solving that applies to any programming field.

2. How Websites Work

Before writing code, understand what happens when you open a website. Some terms first:

Term	Simple meaning
Browser	The program you use to view websites (Chrome, Edge, Firefox, Safari).
Server	A computer that is always on and sends website files to visitors.
Client	Your device and browser. It *asks* the server for things.
URL	The address of a page, for example https://example.com/about
IP address	A number that identifies a computer on the internet, like 93.184.216.34.
DNS	The internet phonebook. It turns a name (example.com) into an IP address.
HTTP / HTTPS	The rules for sending web pages. HTTPS is the encrypted, safer version.

The journey of one page request

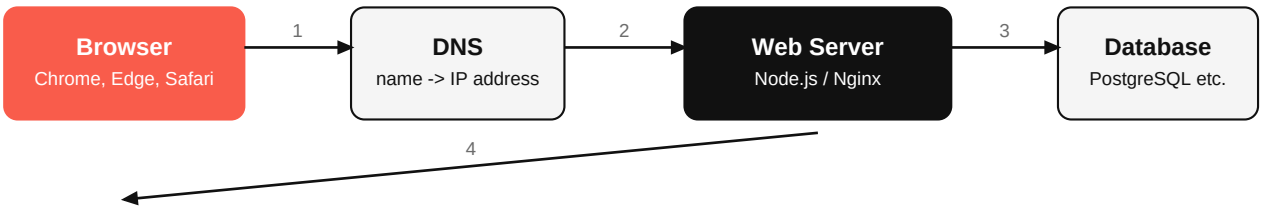


Figure 1.1: What happens when you open a website (diagram)

- **Step 1:** You type a URL. The browser needs the server's address.
- **Step 2:** DNS looks up the IP address for that name.
- **Step 3:** The browser sends an **HTTP request** to the server. If data is needed, the server asks the database.
- **Step 4:** The server sends back a **response** containing HTML, CSS and JavaScript. The browser turns them into the page you see.

HTTP methods and status codes

Method	Used for	Example
GET	Read data	Load a list of courses
POST	Create data	Submit a registration form
PUT / PATCH	Update data	Change your profile photo
DELETE	Remove data	Delete a comment

Status code	Meaning
200	OK: request succeeded
301 / 302	Redirect to another address
400	Bad request: something was wrong in what you sent
401 / 403	Not logged in / not allowed
404	Not found: the page does not exist
500	Server error: a bug on the server

> **Advanced Corner**
> Open Chrome, press **F12** and choose the **Network** tab. Reload any site and click a request. You can see its method, status code, headers and response. This is the best free way to learn how the web really works.

3. Frontend, Backend and Full Stack

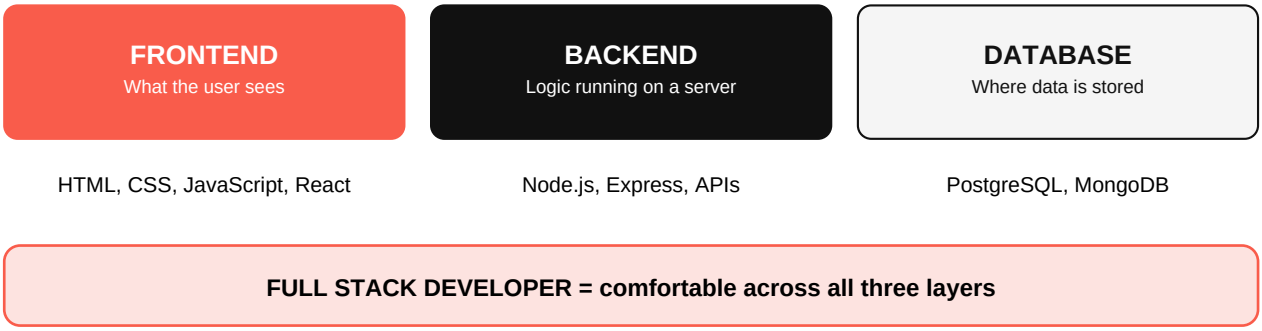


Figure 1.2: The three layers of a web application

	Frontend	Backend	Full Stack
Runs in	The user's browser	A server	Both
Focus	Layout, design, interaction	Logic, security, data handling	Whole application
Core tools	HTML, CSS, JavaScript, React	Node.js, Express, databases, APIs	All of these
Example task	Build a login form	Check the password and create a session	Build both ends of login

Think of a restaurant. The **frontend** is the dining area and menu (what customers see). The **backend** is the kitchen (where the work happens). The **database** is the storeroom. A **full stack developer** can work in all three areas.

4. HTML Fundamentals

HTML (HyperText Markup Language) gives a web page its **structure**. It uses **tags** to mark what each piece of content is: a heading, a paragraph, an image or a link. A tag usually has an opening part and a closing part, like `<p>text</p>`. Together with its content this is called an **element**.

Your first web page

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, Web Developer!</h1>
  <p>This is my first web page.</p>
  <a href="https://developer.mozilla.org">Learn more</a>
</body>
</html>
```

Part	Purpose
<!DOCTYPE html>	Tells the browser this is modern HTML5.
<head>	Information about the page (title, settings). Not shown on the page.
<body>	Everything visible to the visitor.
<h1> to <h6>	Headings, from most to least important.
<p>	A paragraph.
<a href>	A link. The *attribute* href holds the destination.

Essential elements

Images, lists and forms

```
<img src="photo.jpg" alt="Student studying">
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
<form action="/register" method="post">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>
  <button type="submit">Register</button>
</form>
```

Semantic HTML means choosing tags that describe meaning: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` and `<footer>`. It helps search engines and screen readers understand your page.



Figure 1.3: Illustration of a simple page in a browser (drawn diagram, not a screenshot)

Common Mistake
Forgetting the closing tag, or writing `<img>` without the `alt` text. Unclosed tags break layouts; missing alt text hurts accessibility.

Exercise 1
Create a page called **about-me.html** with: a heading with your name, a paragraph about you, a list of three hobbies, one image with alt text and a link to your favourite website. Open it by double-clicking the file.

5. CSS Fundamentals

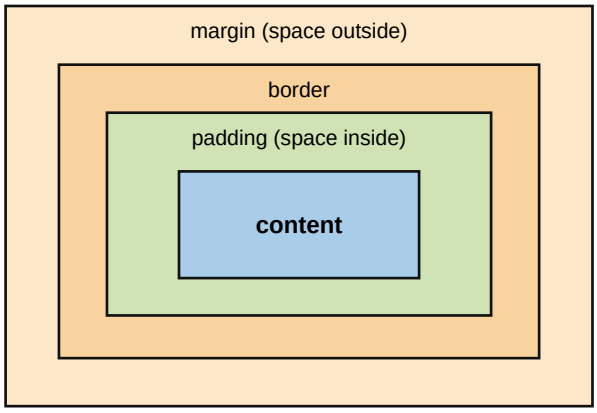
CSS (Cascading Style Sheets) controls how HTML looks: colours, fonts, spacing and layout. A CSS rule has a **selector** (what to style) and **declarations** (how to style it).

styles.css

```css
/* styles.css */
:root {
  --primary: #F95C4B;
}
body {
  font-family: Arial, sans-serif;
  margin: 0;
  color: #111111;
}
.btn {
  background: var(--primary);
  color: white;
  padding: 10px 20px;
  border-radius: 6px;
}
.btn:hover { opacity: 0.85; }
```

Link it in your HTML head: `<link rel="stylesheet" href="styles.css">`. Use `.name` to select a class and `#name` for an id.

The box model



Every HTML element is a box.

Total width = content + padding + border + margin.

Tip: add box-sizing: border-box; so width includes padding and border.

Figure 1.4: The CSS box model

Layout with Flexbox and Grid

Flexbox arranges items in a row or column. **Grid** arranges items in rows and columns together.

Flexbox, Grid and responsive design

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
@media (max-width: 700px) {
  .cards { grid-template-columns: 1fr; }
}
```

The last block is a **media query**: on screens narrower than 700 pixels the three columns become one. This is the heart of **responsive design**, making a site work on phones as well as laptops.

Advanced Corner
Learn **specificity** (which rule wins when two conflict), the `clamp()` function for fluid font sizes, CSS variables for theming, and the `:has()` and container-query features. Read the guides on MDN: https://developer.mozilla.org/en-US/docs/Web/CSS

Exercise 2
Style your about-me page with the SKYROVIX colours (#F95C4B and #111111). Add a navigation bar using Flexbox and make the hobbies appear as three cards using Grid. Test by resizing the browser window.

6. JavaScript Fundamentals

JavaScript (JS) makes pages interactive. It is a real programming language that runs in every browser. HTML is the skeleton, CSS the appearance, and JavaScript the behaviour.

script.js

```
// Variables: containers for values
const name = "Asha";        // cannot be reassigned
let score = 10;             // can change
score = score + 5;

// Functions: reusable blocks of code
function greet(person) {
  return "Hello, " + person + "!";
}
console.log(greet(name));   // Hello, Asha!

// Arrays and loops
const skills = ["HTML", "CSS", "JS"];
skills.forEach(skill => console.log(skill));

// Objects: grouped data
const student = { name: "Asha", age: 20, skills };

// Decisions
if (score >= 15) {
  console.log("Great job");
} else {
  console.log("Keep going");
}
```

Making a page react to the user (the DOM)

The **DOM** (Document Object Model) is the browser's live representation of your HTML. JavaScript can read and change it.

A click counter

```
<button id="btn">Click me</button>
<p id="msg"></p>

<script>
  const btn = document.getElementById("btn");
  const msg = document.getElementById("msg");
  let count = 0;
  btn.addEventListener("click", () => {
    count++;
    msg.textContent = "Clicked " + count + " times";
  });
</script>
```

Asynchronous JavaScript and fetching data

Some tasks take time, such as loading data from a server. **Asynchronous** code lets the page keep working meanwhile. `async/await` is the modern, readable way to write it.

fetch with async/await

```
async function loadUsers() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error("HTTP " + res.status);
    const users = await res.json();
    console.log(users[0].name);
  } catch (err) {
    console.error("Failed:", err.message);
  }
}
loadUsers();
```

Common Mistake
Using = (assign) where you meant === (compare), and forgetting that `fetch` returns a promise that needs `await`.

Exercise 3
Build a to-do list: an input box, an Add button and a list. Each click on Add appends the typed text as a new item. Bonus: add a delete button for each item. Advanced bonus: save items in `localStorage` so they survive a page refresh.

7. React.js Introduction

React is a JavaScript library for building user interfaces from small reusable pieces called **components**. Instead of manually changing the DOM, you describe what the screen should look like for the current data (the **state**), and React updates the page for you. It uses **JSX**, a syntax that looks like HTML inside JavaScript.

App.jsx

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

export default function App() {
  return (
    <main>
      <h1>SKYROVIX Demo</h1>
      <Counter />
    </main>
  );
}
```

Concept	Meaning
Component	A function that returns a piece of UI. Reuse it anywhere.

Concept	Meaning
Props	Inputs passed to a component, like function arguments.
State	Data that changes over time; changing it re-renders the component.
Hook	Special functions like useState and useEffect that add features to components.

Official documentation and tutorial: https://react.dev/learn. Learn plain JavaScript well before React: arrays, objects, functions, `map`, `filter`, destructuring and promises are used constantly.

8. Node.js Introduction

Node.js lets you run JavaScript outside the browser, on your own computer or a server. This means you can use one language for both frontend and backend. **npm** is the package manager installed with Node.js; it downloads ready-made libraries. **Express** is a popular library for building web servers.

Download Node.js (choose the LTS version) from https://nodejs.org, then check it in a terminal:

Terminal

```
node --version
npm --version
mkdir my-api && cd my-api
npm init -y
npm install express
```

server.js

```
const express = require("express");
const app = express();
app.use(express.json());

const courses = [{ id: 1, title: "HTML Basics" }];

app.get("/api/courses", (req, res) => {
  res.json(courses);
});

app.post("/api/courses", (req, res) => {
  const course = { id: courses.length + 1, title: req.body.title };
  courses.push(course);
  res.status(201).json(course);
});

app.listen(3000, () => console.log("Running on port 3000"));
```

Run it with `node server.js` and open http://localhost:3000/api/courses. **localhost** means "this computer" and **3000** is the port, the numbered door the server listens on.

9. APIs and Databases

An **API** (Application Programming Interface) is a menu of requests one program can make to another. A **REST API** uses URLs and HTTP methods: `GET /api/courses` reads courses, `POST /api/courses` creates one. Data is usually exchanged as **JSON**:

JSON example

```
{
  "id": 1,
  "title": "HTML Basics",
  "tags": ["beginner", "frontend"],
  "published": true
}
```

A **database** stores data permanently so it is not lost when a server restarts.

	SQL (relational)	NoSQL (document)
Structure	Tables with rows and columns	Flexible JSON-like documents
Examples	PostgreSQL, MySQL	MongoDB, Firestore
Strong at	Relationships, reports, consistency	Flexible and fast-changing data
Query language	SQL	Varies by product

SQL basics

```
CREATE TABLE students (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT UNIQUE
);
INSERT INTO students (name, email) VALUES ('Asha', 'asha@example.com');
SELECT name, email FROM students WHERE name LIKE 'A%';
```

> **Security basics (Advanced Corner)**
> Never trust user input. Use parameterised queries to prevent SQL injection, hash passwords (never store them as plain text), keep secret keys in environment variables and never commit them to GitHub, and always use HTTPS.

Later parts of this kit cover Supabase (SQL) and Firebase (NoSQL) in depth.

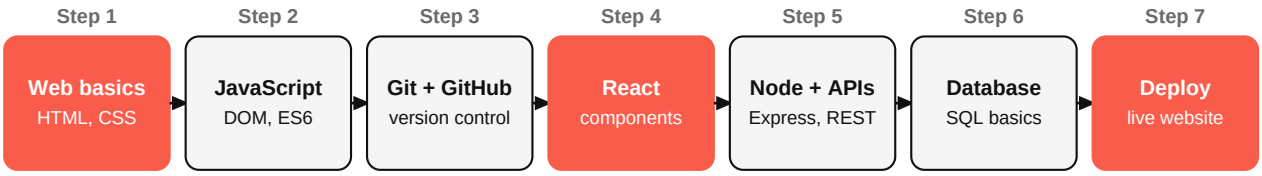
10. Git and GitHub in 5 Minutes

Git records the history of your code so you can undo mistakes and collaborate. **GitHub** is a website that stores Git projects online. Install Git from https://git-scm.com.

Your first push

```
git init
git add .
git commit -m "Add homepage"
git branch -M main
git remote add origin https://github.com/YOUR-NAME/my-site.git
git push -u origin main
```

11. The Full Stack Roadmap



Typical time for a dedicated beginner: 6 to 9 months (about 1 to 2 hours daily). Your pace may differ.

Figure 1.5: A practical learning path

Phase	Learn	Build to prove it
1. Web basics	HTML, CSS, Flexbox, Grid, responsive design	Personal portfolio page
2. JavaScript	Syntax, DOM, events, fetch, ES6+	To-do app, weather app (public API)
3. Tools	VS Code, terminal, Git, GitHub	Push all projects to GitHub
4. React	Components, props, state, hooks, routing	Multi-page React site
5. Backend	Node.js, Express, REST, authentication	Notes API with login
6. Database	SQL or NoSQL, relationships	Student management app
7. Deploy	Hosting, domains, environment variables	Live full stack project with your own link

Capstone project: Student Management App

- **Frontend:** React pages for login, student list, add and edit forms.
- **Backend:** Express API with routes to create, read, update and delete students.
- **Database:** a students table with name, email, course and joining date.
- **Extras:** search, form validation, responsive layout, deployment.

Common Mistakes of beginners
1) Watching tutorials without building. 2) Jumping to React before understanding JavaScript. 3) Never reading error messages (they usually tell you the line and the problem). 4) Not using Git from day one. 5) Comparing yourself to experts instead of your past self.

12. Chapter Assignments and Self-Check

Assignments

- **A1 (beginner):** Build a 3-page static website (Home, About, Contact) with a shared navigation bar.
- **A2 (beginner):** Make it fully responsive and test it at phone width using Chrome DevTools (F12, then the device icon).
- **A3 (intermediate):** Add a JavaScript contact-form validator that checks name, email format and message length.
- **A4 (advanced):** Create a small Express API with GET and POST routes and call it from your page using fetch.

Self-check questions

- What does DNS do?
- Name two differences between frontend and backend.
- What is the difference between `let` and `const`?
- What does a 404 status code mean?
- Why do we use components in React?
- When would you pick SQL over NoSQL?

Official learning resources

Resource	Link	Use it for
MDN Web Docs	https://developer.mozilla.org	HTML, CSS, JavaScript reference and tutorials
React documentation	https://react.dev	Learning React
Node.js	https://nodejs.org	Downloads and docs
Git	https://git-scm.com	Downloads and Pro Git book
GitHub Docs	https://docs.github.com	Using GitHub
VS Code	https://code.visualstudio.com	Editor download and docs
freeCodeCamp	https://www.freecodecamp.org	Free practice curriculum
roadmap.sh	https://roadmap.sh	Visual developer roadmaps

End of Part 01

You now have the map: how the web works, the three layers of an application, the core of HTML, CSS and JavaScript, and a first look at React, Node.js, APIs and databases. The next step is practice. In the following parts of this kit you will set up professional tools and deploy a real project to a live website.

MODULE 02

HTML Complete Guide

Structure, forms, semantics, SEO and accessibility

HTML Complete Guide

2.1 HTML Introduction

HTML stands for HyperText Markup Language. **Hypertext** means text that links to other text. **Markup** means you wrap content in labels (tags) that describe what it is. HTML is not a programming language because it makes no decisions or calculations; it only describes structure. The current standard is maintained as the HTML Living Standard by WHATWG, and MDN documents every element at https://developer.mozilla.org/en-US/docs/Web/HTML.

Real-world use: every page you have ever opened, from a bank login to a school website, is HTML at its core. When a browser receives a page it reads the HTML and builds a tree called the DOM (Document Object Model), which CSS styles and JavaScript changes.

2.2 HTML Document Structure

Standard page skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Short summary for search results">
  <title>Page Title</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
  <script src="script.js" defer></script>
</body>
</html>
```

Line	Why it matters
lang="en"	Tells browsers, translators and screen readers the page language.
charset UTF-8	Lets the page show all characters correctly, including Indian languages.
viewport meta	Without it phones show a zoomed-out desktop layout. Essential for responsive design.
defer on script	Loads the script after the HTML is parsed so elements exist when code runs.

2.3 Elements and Attributes

An **element** is an opening tag, content and a closing tag. An **attribute** adds extra information inside the opening tag as name="value". Some elements are **void** (no content, no closing tag), such as `img`, `br`, `hr` and `input`.

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer">Visit</a>
<img src="logo.png" alt="SKYROVIX logo" width="120" height="40">
<p id="intro" class="lead" title="Hover text">Hello</p>
```

Global attribute	Purpose
id	Unique name for one element (used for links and JavaScript).
class	Group name; many elements can share it (used by CSS).
style	Inline CSS. Avoid; prefer a stylesheet.
hidden	Hides the element.
data-*	Custom data for JavaScript, e.g. data-id="5".

2.4 Headings and Paragraphs

Use `h1` once per page for the main title, then `h2` to `h6` in order, like chapters and sub-chapters. Never choose a heading level for its size; control size with CSS. Use `p` for paragraphs, `strong` for important text, `em` for emphasis, `br` for a line break and `blockquote` for quotations.

2.5 Links and Images

Links and images

```
<!-- Absolute link to another site -->
<a href="https://developer.mozilla.org">MDN</a>
<!-- Relative link to a page in your project -->
<a href="pages/contact.html">Contact</a>
<!-- Link to a section on the same page -->
<a href="#projects">Jump to projects</a>
<!-- Email and phone -->
<a href="mailto:hello@example.com">Email us</a>

<!-- Responsive image with fallback -->
<picture>
  <source srcset="hero.webp" type="image/webp">
  <img src="hero.jpg" alt="Students learning to code" loading="lazy">
</picture>
```

alt text describes an image for blind users and shows if the image fails to load. Decorative images use `alt=""`. Use WebP or AVIF where possible and compress images; large images are the most common cause of slow pages.

2.6 Lists and Tables

Lists and a table

```
<ul><li>HTML</li><li>CSS</li></ul>            <!-- unordered -->
<ol><li>Install</li><li>Build</li></ol>        <!-- ordered -->
<dl><dt>DNS</dt><dd>Internet phonebook</dd></dl>

<table>
  <caption>Weekly timetable</caption>
  <thead>
    <tr><th scope="col">Day</th><th scope="col">Topic</th></tr>
  </thead>
  <tbody>
    <tr><td>Mon</td><td>HTML</td></tr>
    <tr><td>Tue</td><td>CSS</td></tr>
  </tbody>
</table>
```

Common Mistake
Using tables for page layout. Tables are only for data. Use CSS Flexbox or Grid for layout.

2.7 Forms and Input Elements

A complete registration form

```
<form action="/register" method="post">
  <label for="name">Full name</label>
  <input id="name" name="name" type="text" required minlength="2">

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <label for="pw">Password</label>
  <input id="pw" name="password" type="password" minlength="8" required>

  <label for="course">Course</label>
  <select id="course" name="course">
    <option value="web">Web Development</option>
    <option value="ai">AI Basics</option>
  </select>

  <fieldset>
    <legend>Mode</legend>
    <label><input type="radio" name="mode" value="online" checked> Online</label>
    <label><input type="radio" name="mode" value="offline"> Offline</label>
  </fieldset>

  <label><input type="checkbox" name="terms" required> I agree</label>
  <textarea name="msg" rows="4" placeholder="Message"></textarea>
  <button type="submit">Register</button>
</form>
```

Input type	Use
text, email, password	Basic text with built-in checks.
number, range	Numeric entry.
date, time	Browser date and time pickers.

Input type	Use
file	Upload a file.
checkbox, radio	Multiple choice / single choice.
search, tel, url	Keyboard hints on phones.

Browser validation (required, minlength, pattern) is a convenience only. **Always validate again on the server**, because users can bypass the browser.

2.8 Semantic HTML

```
<header><nav>...links...</nav></header>
<main>
  <article>
    <h1>Post title</h1>
    <section><h2>Part one</h2><p>...</p></section>
  </article>
  <aside>Related links</aside>
</main>
<footer>&copy; SKYROVIX</footer>
```

Semantic elements describe meaning. Search engines and screen readers use them to understand the page. Prefer `button` for actions and a for navigation instead of styled `div` elements.

2.9 HTML5 Features

- **audio** and **video** with controls, no plugins needed.
- **canvas** for drawing and simple games; **svg** for sharp scalable graphics.
- **details/summary** for expandable sections without JavaScript.
- **dialog** for modal pop-ups.
- Form helpers: datalist, progress, meter, native validation.

```
<details><summary>What is DNS?</summary>
  <p>It translates names to IP addresses.</p></details>
<video controls width="480" poster="thumb.jpg">
  <source src="intro.mp4" type="video/mp4">
</video>
```

2.10 SEO-Friendly HTML

- Unique, descriptive `title` (about 50-60 characters) and meta description (about 150-160) per page.
- One `h1`, logical heading order.
- Descriptive link text, not "click here".
- Alt text for images; fast-loading images.
- Clean URLs, a sitemap and robots.txt.
- Open Graph tags so links look good when shared.

```
<meta property="og:title" content="SKYROVIX Student Kit">
<meta property="og:description" content="Learn web development">
<meta property="og:image" content="https://example.com/cover.jpg">
```

2.11 Accessibility

Accessibility (a11y) means everyone, including people using screen readers, keyboards only or magnifiers, can use your site. The reference guide is WCAG, published by the W3C (https://www.w3.org/WAI/standards-guidelines/wcag/).

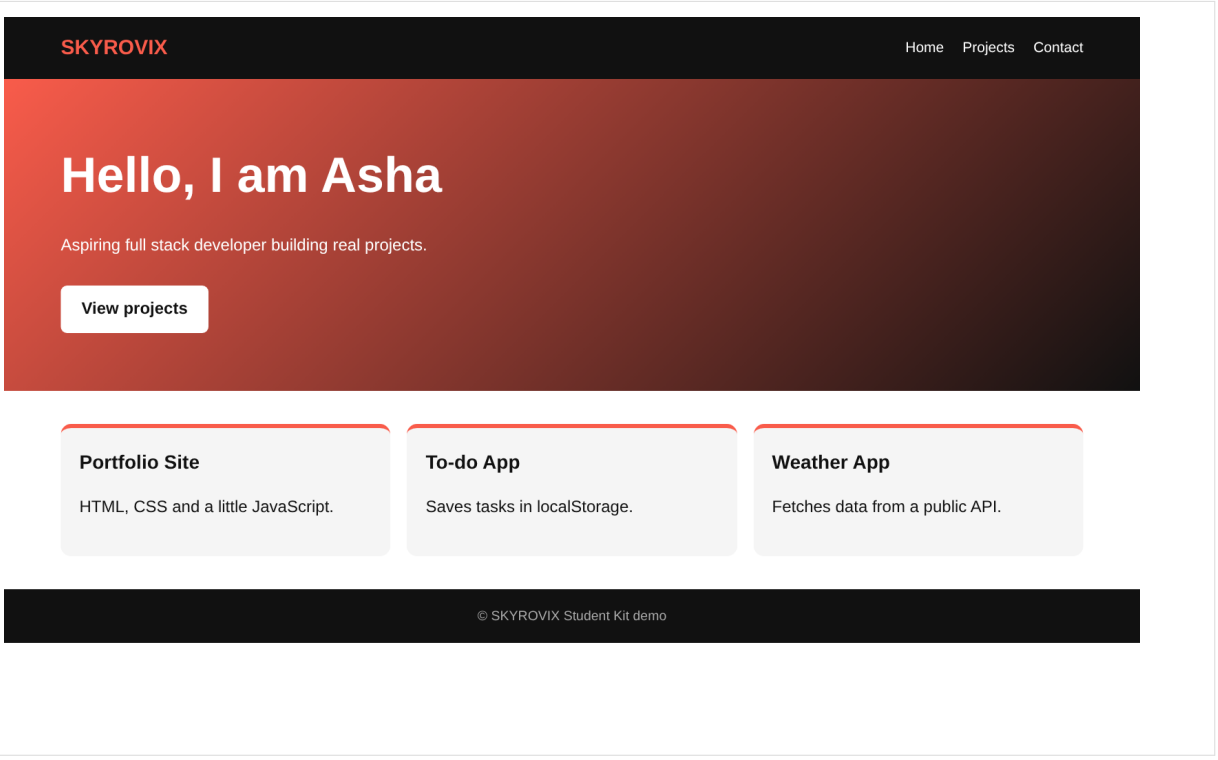
- Every input has a connected `label`.
- Sufficient colour contrast between text and background (WCAG sets minimum ratios).
- Everything works with the keyboard (Tab, Enter, Space); keep visible focus outlines.
- Use ARIA only when native HTML cannot do the job; the first rule of ARIA is to use native elements.
- Provide captions or transcripts for video and audio.

2.12 Practical Projects

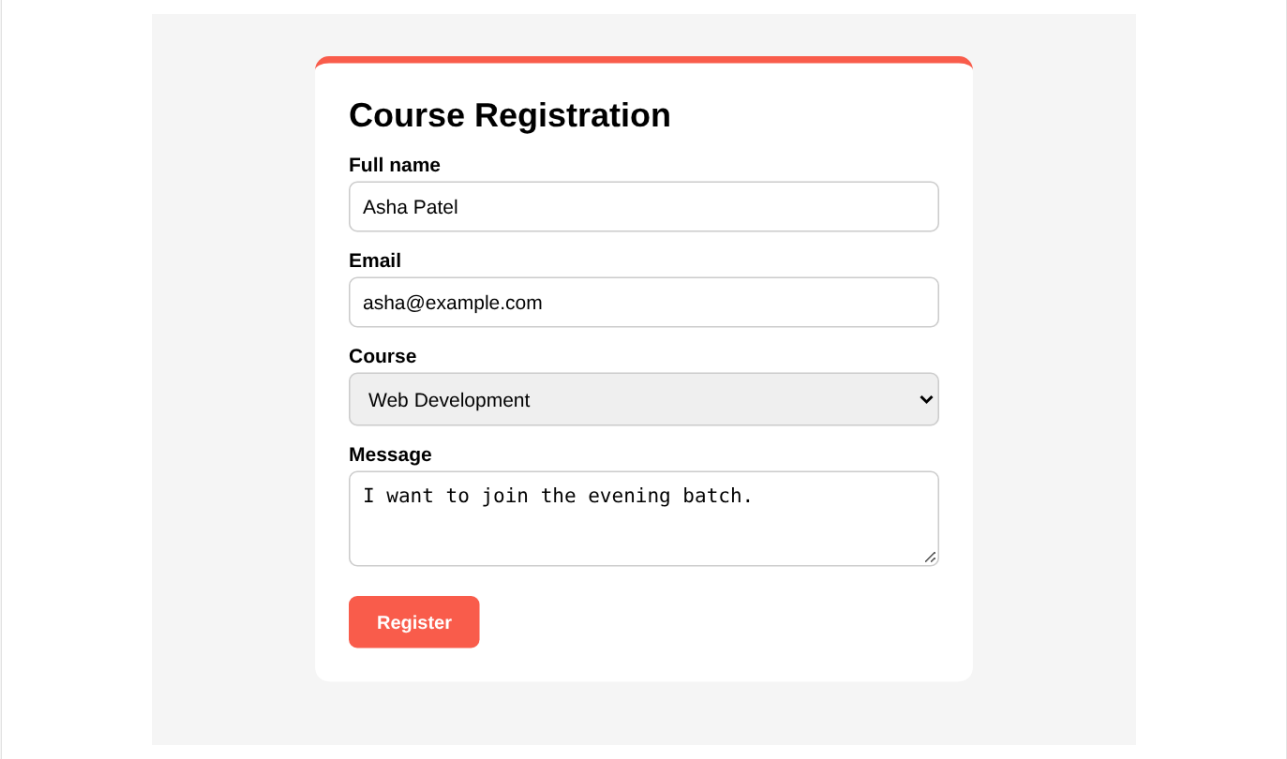
Project A: Resume page. Semantic structure, a skills table, a contact link. **Project B: Course registration form** with the input types above. **Project C: Multi-page site** (Home, Courses, Contact) with shared navigation and a sitemap.

Screenshots: what your pages look like

The images below are real screenshots taken in a Chromium browser from pages written with the code in this module. Your own screen will look the same if you follow along.



Screenshot: portfolio page at desktop width (Chromium)



Screenshot: the registration form from section 2.7

Module 2: Common Mistakes and Troubleshooting

- Missing closing tags or quotes break the layout; validate at https://validator.w3.org.
- Forgetting alt text and labels.
- Wrong relative paths: images do not show. Check folder structure and file name case (logo.PNG is not logo.png on servers).
- Using headings for size instead of structure.

Module 2: Best Practices and Security

- Use semantic elements and keep markup clean.
- Validate user input on the server too; never trust the browser.
- Use rel="noopener noreferrer" with target="_blank".
- Compress images and use lazy loading below the fold.

Module 2: Exercises

Beginner exercises

- Create a page with every heading level and explain why you chose each.
- Build a form with six different input types.
- Make a table of your weekly study plan with caption and header cells.

Advanced exercises

- Make a page that passes the W3C validator with zero errors.
- Run Lighthouse in Chrome DevTools and raise the Accessibility score above 90.
- Add JSON-LD structured data for an organisation (see schema.org).

Mini project

Build a one-page portfolio using header, nav, main with three sections, and footer. Include a contact form and one table.

Module assignment

Build a 4-page website for a fictional college club. Requirements: semantic layout, valid HTML, accessible forms, meta tags on each page, and a short README explaining your choices.

Module 2: Interview Questions

Question	Short model answer
What is the difference between id and class?	id is unique per page; class can be reused on many elements.
Why use semantic HTML?	Meaning for search engines, screen readers and maintainability.
What does the viewport meta tag do?	Makes the layout fit the device width on phones.
Difference between GET and POST in forms?	GET puts data in the URL (searches); POST sends it in the body (logins, submissions).
What is alt text for?	Describes images for users who cannot see them and for failures to load.
Is HTML a programming language?	No, it is a markup language that describes structure.

Module 2: Official Documentation and Resources

Resource	Link
MDN HTML	https://developer.mozilla.org/en-US/docs/Web/HTML
W3C Validator	https://validator.w3.org
WCAG overview	https://www.w3.org/WAI/standards-guidelines/wcag/
web.dev Learn HTML	https://web.dev/learn/html

MODULE 03

CSS Complete Guide

Selectors, layout, responsive design and animation

CSS Complete Guide

3.1 CSS Introduction

CSS (Cascading Style Sheets) tells the browser how HTML should look. **Cascading** means that when several rules apply, a defined order decides the winner. There are three ways to add CSS: **external** file (best), **internal** `style` element, and **inline** style attributes (avoid). Reference: https://developer.mozilla.org/en-US/docs/Web/CSS

3.2 Syntax and Selectors

Selector cheat sheet

```
/* selector { property: value; } */
p { color: #111111; }                /* element */
.card { padding: 16px; }             /* class */
#hero { height: 60vh; }              /* id */
nav a { text-decoration: none; }     /* descendant */
ul > li { margin: 4px 0; }           /* direct child */
h2 + p { margin-top: 0; }            /* adjacent sibling */
a:hover, button:focus-visible { outline: 2px solid #F95C4B; }
input[type="email"] { border: 1px solid #ccc; }  /* attribute */
li:nth-child(odd) { background: #F5F5F5; }       /* pseudo-class */
.btn::after { content: " >"; }                   /* pseudo-element */
```

Specificity: which rule wins?

Selector type	Weight (higher wins)
Inline style	Highest
id (#hero)	High
class, attribute, pseudo-class	Medium
element, pseudo-element	Low
Later rule with equal weight	Wins over earlier rule

Avoid `!important`. If you need it, your selectors are probably too tangled.

3.3 Colours and Backgrounds

```
color: #F95C4B;                 /* hex */
color: rgb(249 92 75);          /* rgb */
color: hsl(5 93% 62%);          /* hsl: hue, saturation, lightness */
background: linear-gradient(135deg, #F95C4B, #111111);
background-image: url("pattern.png");
background-size: cover;
background-position: center;
opacity: 0.9;
```

3.4 Typography

```
body {
  font-family: "Inter", system-ui, Arial, sans-serif;
  font-size: 1rem;      /* 16px by default */
  line-height: 1.6;
}
h1 { font-size: clamp(2rem, 5vw, 3.5rem); }  /* fluid size */
p  { max-width: 65ch; }                      /* readable line length */
```

Units: **px** is fixed; **rem** scales with the root font size (good for accessibility); **em** scales with the parent; **%** is relative to the parent; **vw/vh** are percentages of the viewport. For web fonts, Google Fonts (https://fonts.google.com) provides free fonts with embed instructions; always set fallback fonts.

3.5 Box Model

Every element is a box: content, padding, border, margin (see Figure 1.4 in Module 1). By default width covers only content, which surprises beginners. Fix it globally:

```
*, *::before, *::after { box-sizing: border-box; }
.card {
  width: 300px;             /* now includes padding and border */
  padding: 16px;
  border: 1px solid #ddd;
  margin: 0 auto;           /* centre a block horizontally */
}
```

3.6 Margin and Padding

```
margin: 10px;                /* all sides */
margin: 10px 20px;           /* top-bottom, left-right */
margin: 10px 20px 30px 40px; /* top right bottom left (clockwise) */
padding-inline: 1rem;        /* left and right (logical) */
```

Margin collapse: vertical margins of neighbouring blocks merge into the larger one rather than adding up. Use padding, gap or Flexbox/Grid to avoid surprises.

3.7 Flexbox

Flexbox

```
.row {
  display: flex;
  flex-direction: row;          /* or column */
  justify-content: space-between; /* main axis */
  align-items: center;            /* cross axis */
  gap: 16px;
  flex-wrap: wrap;
}
.row .grow { flex: 1; }           /* take remaining space */
```

Property	What it does
justify-content	Spacing along the main axis (start, center, space-between, space-around).
align-items	Alignment on the cross axis.
gap	Space between items.

Property	What it does
flex: 1	Item grows to fill space.
order	Change visual order (keep accessibility in mind).

3.8 CSS Grid

Grid

```
.layout {
  display: grid;
  grid-template-columns: 240px 1fr;   /* sidebar + content */
  grid-template-areas:
    "header header"
    "side   main"
    "footer footer";
  gap: 16px;
}
header { grid-area: header; }
aside  { grid-area: side; }
main   { grid-area: main; }
footer { grid-area: footer; }

.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;   /* responsive without media queries */
}
```

Rule of thumb: Flexbox for one dimension (a row or a column), Grid for two dimensions (rows and columns together). They are often combined.

3.9 Positioning

Value	Behaviour
static	Default; normal flow.
relative	Normal flow, but can be nudged; becomes the reference for absolute children.
absolute	Removed from flow, placed relative to nearest positioned ancestor.
fixed	Stays in place relative to the viewport (e.g. a floating button).
sticky	Scrolls normally until a threshold, then sticks (e.g. a header).

```
.header { position: sticky; top: 0; z-index: 10; background: white; }
.badge  { position: absolute; top: -6px; right: -6px; }
.card   { position: relative; }
```

3.10 Transitions and Animations

```
.btn { transition: transform .2s ease, background .2s ease; }
.btn:hover { transform: translateY(-2px); }

@keyframes fadeUp {
  from { opacity: 0; transform: translateY(12px); }
  to   { opacity: 1; transform: none; }
}
.hero { animation: fadeUp .6s ease both; }

@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}
```

Animate `transform` and `opacity` because they are cheap for the browser. Respect users who prefer reduced motion.

3.11 Responsive Web Design

Responsive design makes one site work on phones, tablets and desktops. Use a **mobile-first** approach: write styles for small screens first, then add rules for larger screens. Use flexible units, flexible images (`max-width: 100%`) and flexible layouts (Flexbox, Grid).

3.12 Media Queries

```
/* mobile first: base styles are for phones */
.cards { display: grid; gap: 16px; grid-template-columns: 1fr; }

@media (min-width: 640px)  { .cards { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 1024px) { .cards { grid-template-columns: repeat(3, 1fr); } }

@media (prefers-color-scheme: dark) {
  :root { --bg: #111111; --text: #ffffff; }
}
```

3.13 CSS Variables

```
:root {
  --primary: #F95C4B;
  --black: #111111;
  --light: #F5F5F5;
  --radius: 8px;
  --space: 16px;
}
.btn { background: var(--primary); border-radius: var(--radius); padding: var(--space); }
```

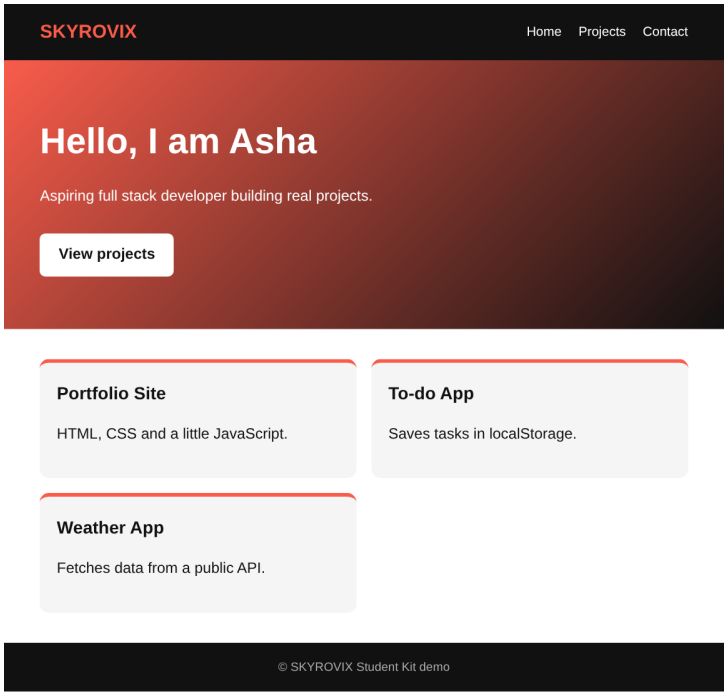
Variables make themes and brand changes easy: change one line and the whole site updates. They can be changed with JavaScript too.

3.14 Practical Projects

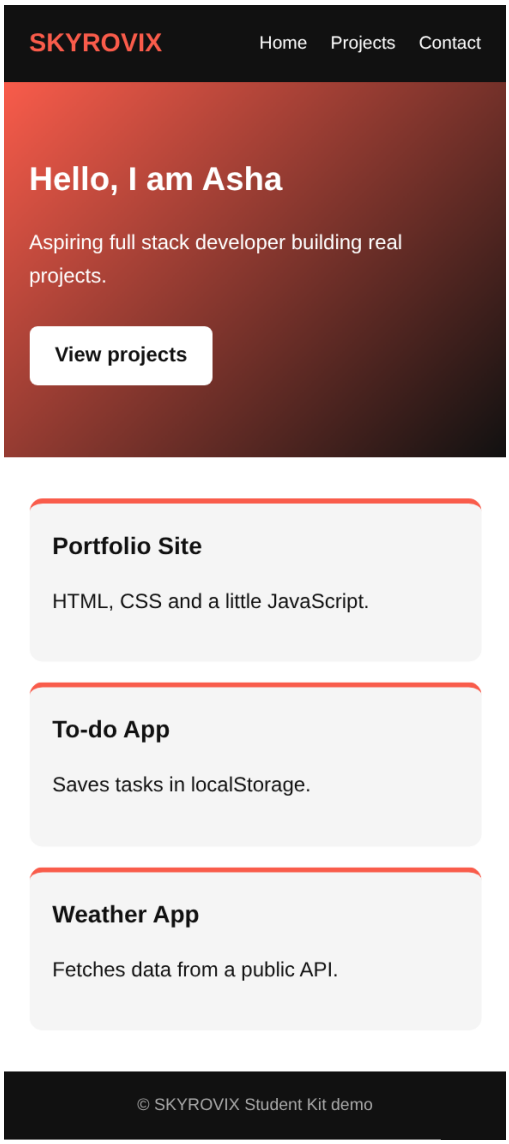
Project A: Responsive navbar that collapses on phones. **Project B: Pricing page** with three cards using Grid. **Project C: Landing page** with hero, features, testimonials and footer, using the SKYROVIX variables above.

Screenshots: responsive layout in action

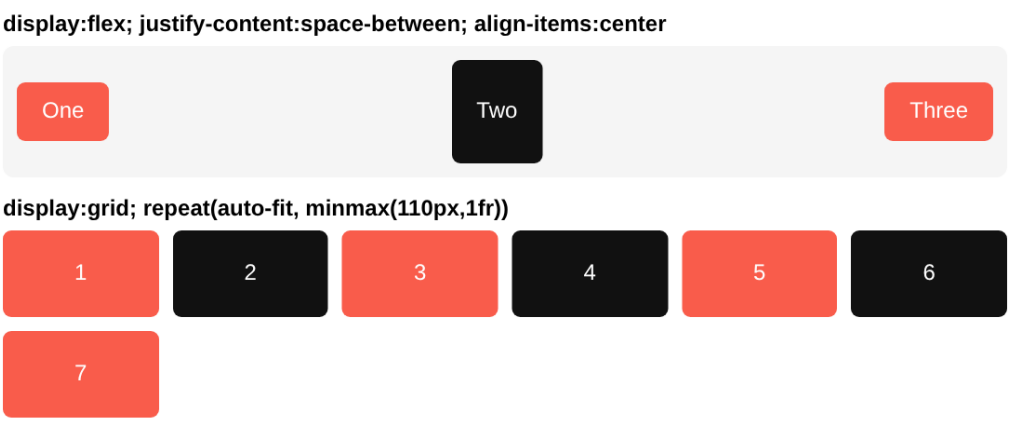
The same HTML and CSS rendered at three widths. Notice how the three cards become two, then one, as the screen narrows. This is the media query in section 3.12 at work.



Screenshot: tablet width (768px), two columns



Screenshot: phone width (390px), one column, full page



Screenshot: Flexbox row (top) and an auto-fit Grid (bottom)

Module 3: Common Mistakes and Troubleshooting

- Stylesheet not applying: wrong path in the link tag or a typo in the selector. Inspect the element in DevTools (F12) to see which rules apply.
- Forgetting box-sizing: border-box, causing overflow.
- Using fixed widths that break on phones.
- Overusing !important.
- Centring confusion: text-align centres text; margin: 0 auto centres a block; Flexbox/Grid centre items.

Module 3: Best Practices and Security

- Keep a small design system of variables for colours, spacing and fonts.
- Name classes by purpose (card, btn-primary), not appearance.
- Test at several widths and with browser zoom at 200 percent.
- Check colour contrast; do not rely on colour alone.
- Remove unused CSS before production.

Module 3: Exercises

Beginner exercises

- Style a button with normal, hover, focus and disabled states.
- Centre a div both horizontally and vertically with Flexbox.
- Build a 3-column layout that becomes 1 column on phones.

Advanced exercises

- Rebuild a layout from a screenshot using Grid areas.
- Create a dark mode toggle using CSS variables and a class on the html element.
- Make a CSS-only accordion or tabs and explain its accessibility limits.

Mini project

Build a responsive landing page for a fictional product using the SKYROVIX colours with hero, three feature cards and a footer.

Module assignment

Convert your Module 2 four-page site into a fully responsive, styled site with a shared stylesheet, CSS variables, hover effects and a reduced-motion rule. Submit screenshots at phone, tablet and desktop widths.

Module 3: Interview Questions

Question	Short model answer
What is the box model?	Content, padding, border and margin around every element.
Flexbox vs Grid?	Flexbox is one-dimensional; Grid is two-dimensional.
What is specificity?	The rule that decides which conflicting declaration wins.
What does mobile-first mean?	Write base styles for small screens, then enhance with min-width queries.
rem vs em?	rem relates to the root size; em to the parent size.
How do you centre a div?	margin: 0 auto for width-limited block, or Flexbox/Grid centring.

Module 3: Official Documentation and Resources

Resource	Link
MDN CSS	https://developer.mozilla.org/en-US/docs/Web/CSS
web.dev Learn CSS	https://web.dev/learn/css
CSS Tricks Flexbox guide	https://css-tricks.com/snippets/css/a-guide-to-flexbox/
Can I use	https://caniuse.com

MODULE 04

JavaScript Complete Guide

From variables to asynchronous programming

JavaScript Complete Guide

4.1 JavaScript Introduction

JavaScript is the programming language of the web. It runs in browsers and, through Node.js, on servers. It is **dynamically typed** (variables can hold any type) and **single-threaded** (one thing at a time, with an event system for waiting tasks). The language standard is ECMAScript; the best reference is https://developer.mozilla.org/en-US/docs/Web/JavaScript.

```
<script src="script.js" defer></script>

// script.js
console.log("Hello from JavaScript");
```

Open Chrome DevTools (F12) and the **Console** tab to see output and errors. It is also a free place to experiment.

4.2 Variables and Data Types

```
const school = "SKYROVIX";    // string, cannot be reassigned
let age = 20;                 // number
let isStudent = true;         // boolean
let nothing = null;           // intentional empty value
let notSet;                   // undefined (no value yet)
const id = 9007199254740993n;// bigint
const key = Symbol("id");     // symbol (rare for beginners)

console.log(typeof age);      // "number"
console.log(typeof null);     // "object" (a famous historical quirk)
```

Use `const` by default and `let` when the value must change. Avoid `var`; it has confusing scope rules. Template strings: `` `Hello ${name}` ``.

4.3 Operators

Group	Operators	Notes
Arithmetic	+ - * / % **	% is remainder; ** is power.
Comparison	=== !== < > <= >=	Use === (strict) not ==.
Logical	&& || !	Also used for short-circuit defaults.
Assignment	= += -= *=	x += 1 means x = x + 1.
Nullish	?? ?.	a ?? b uses b only if a is null/undefined; a?.b safely reads b.

```
console.log(0.1 + 0.2);         // 0.30000000000000004 (floating-point)
console.log("5" + 1);           // "51" (string joining)
console.log("5" - 1);           // 4   (numeric)
console.log(0 == false);        // true  (loose equality surprises)
console.log(0 === false);       // false (strict equality is safer)
const name = user?.profile?.name ?? "Guest";
```

4.4 Conditions and Loops

```
if (marks >= 90) { grade = "A"; }
else if (marks >= 75) { grade = "B"; }
else { grade = "C"; }

const label = marks >= 40 ? "Pass" : "Fail";   // ternary

switch (day) {
  case "Sat": case "Sun": console.log("Weekend"); break;
  default: console.log("Weekday");
}

for (let i = 1; i <= 5; i++) console.log(i);
let n = 3; while (n > 0) { n--; }
for (const item of ["a","b"]) console.log(item);      // values
for (const key in { x: 1, y: 2 }) console.log(key);   // keys
```

Falsy values that count as false in conditions: false, 0, "", null, undefined, NaN. Everything else is truthy, including "0" and empty arrays.

4.5 Functions

```
function add(a, b = 0) { return a + b; }          // declaration with default
const mul = function (a, b) { return a * b; };    // expression
const square = x => x * x;                        // arrow function
const sumAll = (...nums) => nums.reduce((t, n) => t + n, 0);  // rest

function counter() {
  let c = 0;                    // closure: inner function remembers c
  return () => ++c;
}
const next = counter();
next(); next();                 // 1, 2
```

A **closure** is a function that remembers the variables around it when it was created. Functions are values: you can pass them as arguments (**callbacks**) and return them.

4.6 Arrays and Objects

```
const nums = [5, 2, 9];
nums.push(7); nums.pop(); nums.length;
nums.map(n => n * 2);             // [10, 4, 18] new array
nums.filter(n => n > 3);          // [5, 9]
nums.find(n => n > 5);            // 9
nums.reduce((t, n) => t + n, 0);  // 16
nums.includes(2);                 // true
[...nums].sort((a, b) => a - b);  // copy then sort numbers

const student = { name: "Asha", skills: ["HTML", "CSS"], address: { city: "Pune" } };
const { name, skills } = student;          // destructuring
const copy = { ...student, name: "Ravi" };  // spread (shallow copy)
Object.keys(student); Object.entries(student);
JSON.stringify(student); JSON.parse('{"a":1}');
```

Common Mistake
Sorting numbers without a compare function: [10, 9, 1].sort() gives [1, 10, 9] because it compares as text. Also remember that sort() changes the original array.

4.7 DOM Manipulation

The **DOM** is a tree of objects representing the page. JavaScript selects elements, changes them, creates new ones and removes them.

```
const title = document.querySelector("h1");
const items = document.querySelectorAll(".item");
title.textContent = "Updated";
title.classList.add("active");
title.style.color = "#F95C4B";

const li = document.createElement("li");
li.textContent = "New task";
document.querySelector("ul").appendChild(li);
li.remove();
document.getElementById("img").setAttribute("alt", "Description");
```

Use `textContent` to insert text. Avoid `innerHTML` with user-provided text because it can allow **XSS** (cross-site scripting) attacks.

4.8 Events

```
const form = document.querySelector("form");
form.addEventListener("submit", (e) => {
  e.preventDefault();                    // stop page reload
  const email = form.elements.email.value.trim();
  if (!email.includes("@")) { alert("Enter a valid email"); return; }
  console.log("Submitting", email);
});

// Event delegation: one listener for many items
document.querySelector("ul").addEventListener("click", (e) => {
  if (e.target.matches("button.delete")) e.target.closest("li").remove();
});
```

Common events: click, input, change, submit, keydown, mouseover, scroll, DOMContentLoaded, load. **Bubbling** means events travel up from the clicked element to its parents; delegation uses this.

4.9 ES6+ Features

- let and const, template strings, arrow functions.
- Destructuring, spread and rest.
- Modules: `export` and `import`.
- Classes, optional chaining, nullish coalescing.
- Promises, async/await, Map, Set.

```
// math.js
export const PI = 3.14159;
export default function area(r) { return PI * r * r; }

// main.js (use <script type="module"> in the browser)
import area, { PI } from "./math.js";

class Student {
  constructor(name) { this.name = name; }
  greet() { return `Hi, ${this.name}`; }
}
```

4.10 Promises and Async/Await

A **promise** represents a value that will arrive later. It is pending, then either fulfilled or rejected. JavaScript keeps the page responsive using the **event loop**: slow tasks wait in the background and their callbacks run later.

```
const wait = (ms) => new Promise(res => setTimeout(res, ms));

async function run() {
  console.log("start");
  await wait(1000);
  console.log("after 1 second");
}

// run tasks in parallel
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

4.11 Fetch API

```
async function createStudent(data) {
  const res = await fetch("/api/students", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
  if (!res.ok) throw new Error(`Server said ${res.status}`);
  return res.json();
}
```

Important: `fetch` does not reject on HTTP errors like 404 or 500; check `res.ok` yourself. Browsers block requests to other sites unless the server allows them (**CORS**).

4.12 Error Handling

```
try {
  const data = JSON.parse(userInput);
} catch (err) {
  console.error("Bad JSON:", err.message);
} finally {
  console.log("always runs");
}
throw new Error("Something went wrong");
```

4.13 Local Storage

```
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme");
localStorage.setItem("tasks", JSON.stringify(tasks));
const saved = JSON.parse(localStorage.getItem("tasks") || "[]");
localStorage.removeItem("theme");
```

localStorage keeps text data in the browser (limited to a few megabytes) and persists after closing. sessionStorage clears when the tab closes. **Never store passwords or tokens that grant account access there** if you can avoid it; any script on the page can read them.

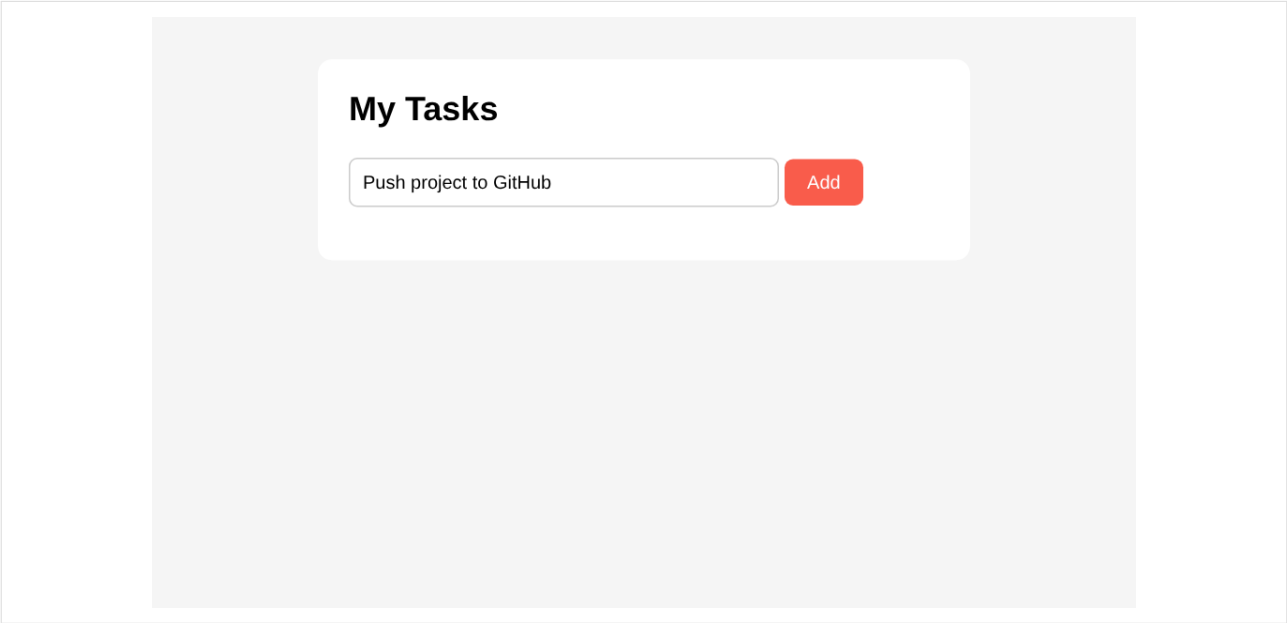
4.14 Practical Projects

Complete to-do app with saving

```
<input id="task" placeholder="New task">
<button id="add">Add</button>
<ul id="list"></ul>
<script>
  let tasks = JSON.parse(localStorage.getItem("tasks") || "[]");
  const list = document.getElementById("list");
  function render() {
    list.innerHTML = "";
    tasks.forEach((t, i) => {
      const li = document.createElement("li");
      li.textContent = t + " ";
      const del = document.createElement("button");
      del.textContent = "x";
      del.onclick = () => { tasks.splice(i, 1); save(); };
      li.appendChild(del); list.appendChild(li);
    });
  }
  function save() { localStorage.setItem("tasks", JSON.stringify(tasks)); render(); }
  document.getElementById("add").onclick = () => {
    const v = document.getElementById("task").value.trim();
    if (v) { tasks.push(v); save(); }
  };
  render();
</script>
```

Other projects: a tip calculator, a quiz app with score, a weather app using a public API with a free key (keep keys out of public repos), and an expense tracker with charts.

Screenshot: the finished to-do app



Screenshot: to-do app from section 4.14 after adding three tasks

Module 4: Common Mistakes and Troubleshooting

- "undefined is not a function" or "Cannot read properties of undefined": print the variable with console.log and check spelling and the data shape.
- Script runs before the HTML exists: add defer or put it at the end of body.
- Forgetting await on async calls and getting a Promise instead of data.
- Using == instead of ===.
- Changing arrays while looping through them.

Module 4: Best Practices and Security

- Use const by default, name things clearly, keep functions small.
- Never trust user input; sanitise before inserting into the page.
- Do not put secrets or API keys in frontend code; anyone can read them.
- Handle errors in every network call and show friendly messages.
- Format code with Prettier and lint with ESLint.

Module 4: Exercises

Beginner exercises

- Write a function that returns the largest number in an array.
- Reverse a string and check whether it is a palindrome.
- Build a button that toggles dark mode and remembers the choice.

Advanced exercises

- Implement debounce for a search box and explain how it works.
- Fetch a public API, show results in cards, and add loading and error states.
- Write your own versions of map and filter using loops.

Mini project

Build a quiz app with five questions, score tracking, and a result screen.

Module assignment

Build a complete expense tracker: add, delete and total expenses, filter by category, save in localStorage, and display a summary. Submit the code on GitHub with a README.

Module 4: Interview Questions

Question	Short model answer
var vs let vs const?	var is function-scoped and hoisted; let/const are block-scoped; const cannot be reassigned.
What is a closure?	A function that remembers variables from where it was created.
== vs ===?	== converts types before comparing; === does not.
What is the event loop?	The mechanism that runs queued callbacks when the call stack is empty.
What is a promise?	An object for a value available later, fulfilled or rejected.
What is event delegation?	Handling events for many children with one listener on a parent.
What is XSS?	Injecting malicious scripts into a page; prevent with proper escaping and avoiding innerHTML with untrusted text.

Module 4: Official Documentation and Resources

Resource	Link
MDN JavaScript	https://developer.mozilla.org/en-US/docs/Web/JavaScript
javascript.info	https://javascript.info
ECMAScript spec	https://tc39.es/ecma262/
Node.js docs	https://nodejs.org/docs/latest/api/

MODULE 05

Frontend Development

React, JSX, hooks, routing, TypeScript, Next.js and Tailwind CSS

Frontend Development

5.1 Frontend Architecture

A modern frontend is more than HTML files. It is a **project** with source code, a **build tool** that bundles and optimises it, **packages** from npm, and **components** that split the interface into reusable parts. Two common styles exist: **Single Page Applications (SPA)**, where one HTML page is loaded and JavaScript swaps views (React with Vite), and **server-rendered apps**, where the server prepares HTML for each request (Next.js). SPAs feel app-like; server rendering helps first load speed and search engines.

Layer	Responsibility	Example
UI components	Display data	Button, Card, Navbar
State	Data that changes	Cart items, form values
Routing	Which page shows for which URL	/courses/5
Data access	Talk to APIs	fetch("/api/courses")
Styling	Appearance	CSS, Tailwind

Typical React folder structure

```
my-app/
  index.html
  package.json
  src/
    main.jsx          entry point
    App.jsx           root component
    components/       Navbar.jsx, CourseCard.jsx
    pages/            Home.jsx, Courses.jsx
    hooks/            useFetch.js
    api/              courses.js
```

5.2 React.js Complete Guide

React builds interfaces from **components**. Your job is to describe the screen for the current **state**; React updates the real page efficiently. Create a project with Vite (a fast build tool; see https://vite.dev):

Terminal

```
npm create vite@latest skyrovix-app -- --template react
cd skyrovix-app
npm install
npm run dev
```

Open the local address printed in the terminal. Edit `src/App.jsx`; the browser updates on save. The official tutorial is at https://react.dev/learn.

5.3 JSX

JSX is a syntax that lets you write markup inside JavaScript. Browsers cannot read it, so the build tool converts it. Rules: return one parent element (or a fragment), close every tag, use `className` instead of class, and put JavaScript inside curly braces.

Welcome.jsx

```jsx
const name = "Asha";
const courses = ["HTML", "CSS", "React"];

export default function Welcome() {
  return (
    <>
      <h1 className="title">Hello, {name.toUpperCase()}</h1>
      <p>You have {courses.length} courses.</p>
      <ul>
        {courses.map(c => <li key={c}>{c}</li>)}
      </ul>
      {courses.length > 2 && <p>Great progress!</p>}
    </>
  );
}
```

Lists need a stable `key` on each item (use an id, not the array index, when items can be reordered). Conditional display uses `&&` or the ternary operator.

5.4 Components and Props

A component is a function whose name starts with a capital letter. **Props** are the inputs you pass in. Props are read-only: a component must not change them.

Reusable component

```jsx
function CourseCard({ title, level = "Beginner", onSelect }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <span>{level}</span>
      <button onClick={() => onSelect(title)}>Enroll</button>
    </div>
  );
}

function Courses() {
  const handle = (t) => alert("Enrolled in " + t);
  return (
    <div className="grid">
      <CourseCard title="HTML" onSelect={handle} />
      <CourseCard title="React" level="Intermediate" onSelect={handle} />
    </div>
  );
}
```

children is a special prop for content placed between tags: `<Card>anything</Card>` makes it available as `props.children`. Prefer composition (putting components inside components) over copying code.

5.5 State and Hooks

State is memory inside a component. When state changes, React runs the component again and updates the screen. **Hooks** are functions starting with `use` that add features. Rules: call hooks only at the top level of a component (not in loops or conditions) and only in components or custom hooks.

Search.jsx

```jsx
import { useState, useEffect, useRef } from "react";

function Search() {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const inputRef = useRef(null);

  useEffect(() => { inputRef.current.focus(); }, []);   // once on load

  useEffect(() => {
    if (!query) { setResults([]); return; }
    const controller = new AbortController();
    setLoading(true);
    fetch("/api/courses?q=" + encodeURIComponent(query), { signal: controller.signal })
      .then(r => r.json())
      .then(setResults)
      .catch(() => {})
      .finally(() => setLoading(false));
    return () => controller.abort();                    // cleanup
  }, [query]);                                          // re-run when query changes

  return (
    <div>
      <input ref={inputRef} value={query} onChange={e => setQuery(e.target.value)} />
      {loading && <p>Loading...</p>}
      <ul>{results.map(r => <li key={r.id}>{r.title}</li>)}</ul>
    </div>
  );
}
```

Hook	Use it for
useState	Values that change and affect the screen.
useEffect	Side effects: fetching, timers, subscriptions. The dependency array controls when it re-runs.
useRef	Reach a DOM element or keep a value without re-rendering.
useMemo / useCallback	Optimisation. Add only when you measured a problem.
useContext	Read shared data from a Context (section 5.7).
useReducer	Complex state with many update types.

Common Mistakes
1) Changing state directly (`items.push(x)`) instead of creating a new array (`setItems([...items, x])`). 2) Forgetting the dependency array and causing an infinite loop. 3) Using state for data you can calculate from other state.

Forms in React

Controlled form

```
function Register() {
  const [form, setForm] = useState({ name: "", email: "" });
  const [error, setError] = useState("");

  const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });
  const submit = (e) => {
    e.preventDefault();
    if (!form.email.includes("@")) return setError("Enter a valid email");
    setError("");
    console.log("Send to API", form);
  };
  return (
    <form onSubmit={submit}>
      <input name="name" value={form.name} onChange={change} />
      <input name="email" value={form.email} onChange={change} />
      {error && <p role="alert">{error}</p>}
      <button>Register</button>
    </form>
  );
}
```

5.6 React Router

A SPA needs routing so the URL changes without a full reload. React Router is the most common library (docs: https://reactrouter.com). Install it with `npm install react-router-dom`. Recent major versions reorganised packages, so always follow the installation page for the version you install.

App.jsx with routes

```
import { BrowserRouter, Routes, Route, Link, useParams, Navigate } from "react-router-dom";

function CourseDetail() {
  const { id } = useParams();          // from /courses/:id
  return <h2>Course #{id}</h2>;
}

export default function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> <Link to="/courses">Courses</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/courses" element={<Courses />} />
        <Route path="/courses/:id" element={<CourseDetail />} />
        <Route path="/old" element={<Navigate to="/courses" replace />} />
        <Route path="*" element={<h2>404 Not Found</h2>} />
      </Routes>
    </BrowserRouter>
  );
}
```

Use `Link`, not plain `a`, for internal navigation. Protected pages are done by checking the user and rendering `Navigate` to the login page when not signed in. When you deploy an SPA, configure a rewrite to index.html so refreshing /courses does not give a 404 (see Module 16).

5.7 Context API

Passing props through many layers is called **prop drilling**. Context shares data (theme, logged-in user) with any component below a provider.

AuthContext.jsx

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (u) => setUser(u);
  const logout = () => setUser(null);
  return <AuthContext.Provider value={{ user, login, logout }}>{children}</AuthContext.Provid
er>;
}
export const useAuth = () => useContext(AuthContext);

// usage in any component
function Navbar() {
  const { user, logout } = useAuth();
  return user ? <button onClick={logout}>Logout {user.name}</button> : <a href="/login">Login
</a>;
}
```

Context re-renders every consumer when its value changes, so keep it for data that changes rarely. For large apps, libraries such as Zustand or Redux Toolkit manage state; learn plain React first.

5.8 API Integration

Wrap fetch logic in a reusable **custom hook** so every page handles loading and errors the same way.

useFetch.js and a page

```
import { useState, useEffect } from "react";

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let ignore = false;
    setLoading(true);
    fetch(url)
      .then(r => { if (!r.ok) throw new Error("HTTP " + r.status); return r.json(); })
      .then(d => { if (!ignore) setData(d); })
      .catch(e => { if (!ignore) setError(e.message); })
      .finally(() => { if (!ignore) setLoading(false); });
    return () => { ignore = true; };
  }, [url]);

  return { data, error, loading };
}

function Students() {
  const { data, error, loading } = useFetch(import.meta.env.VITE_API_URL + "/students");
  if (loading) return <p>Loading...</p>;
  if (error) return <p role="alert">Error: {error}</p>;
  return <ul>{data.map(s => <li key={s.id}>{s.name}</li>)}</ul>;
}
```

`import.meta.env` holds Vite environment variables; only those starting with `VITE_` reach the browser. Never put secrets there. For caching and refetching in bigger apps, TanStack Query (https://tanstack.com/query) is a popular choice.

5.9 TypeScript Fundamentals

TypeScript is JavaScript plus **types**: labels that describe what kind of value a variable or function expects. Your editor then warns about mistakes before you run the code. TypeScript is compiled to JavaScript. Docs: https://www.typescriptlang.org/docs/

Types, interfaces and a typed component

```
let age: number = 20;
let name: string = "Asha";
let skills: string[] = ["HTML", "CSS"];
let id: string | number = 5;             // union type

interface Student {
  id: number;
  name: string;
  email?: string;                        // optional
}
type Role = "admin" | "teacher" | "student";

function greet(s: Student, role: Role = "student"): string {
  return `Hello ${s.name} (${role})`;
}

// React with TypeScript (file ends in .tsx)
type CardProps = { title: string; level?: "Beginner" | "Advanced" };
export function Card({ title, level = "Beginner" }: CardProps) {
  return <div><h3>{title}</h3><small>{level}</small></div>;
}
```

Create a TypeScript React project with `npm create vite@latest my-app -- --template react-ts`. Start with simple types and let TypeScript infer the rest; avoid overusing `any`, which turns checking off.

5.10 Next.js Introduction

Next.js (https://nextjs.org/docs) is a React framework from Vercel that adds file-based routing, server rendering, API routes and optimisations. Create a project:

Terminal

```
npx create-next-app@latest skyrovix-next
cd skyrovix-next
npm run dev
```

The installer asks questions (TypeScript, ESLint, Tailwind, App Router). Defaults change between versions, so read the prompts. This guide uses the **App Router**, the recommended routing system in current Next.js.

5.11 Next.js Routing

In the App Router, folders inside `app/` become URLs and a file named `page.tsx` makes the route visible.

File-based routing

```
app/
  layout.tsx            shared layout (navbar, footer) around all pages
  page.tsx              /
  about/page.tsx        /about
  courses/page.tsx      /courses
  courses/[id]/page.tsx /courses/5        (dynamic segment)
  loading.tsx           shown while a page loads
  not-found.tsx         custom 404
```

Dynamic route

```
// app/courses/[id]/page.tsx
type Props = { params: Promise<{ id: string }> };

export default async function CoursePage({ params }: Props) {
  const { id } = await params;       // params is a Promise in recent versions
  const res = await fetch(`https://api.example.com/courses/${id}`);
  const course = await res.json();
  return <h1>{course.title}</h1>;
}
```

Navigate with the `Link` component from `next/link`. Because Next.js versions change how things like `params` work, check the docs for your installed version.

5.12 Server and Client Components

In the App Router, components are **Server Components** by default: they run on the server, can fetch data directly and send ready HTML, so less JavaScript reaches the browser. Components that need interactivity (state, effects, click handlers, browser APIs) must be **Client Components**, marked with `"use client"` at the top of the file.

Server and client components

```
// app/courses/page.tsx   (Server Component: no hooks, can use async/await)
import LikeButton from "./LikeButton";

export default async function Courses() {
  const courses = await fetch("https://api.example.com/courses").then(r => r.json());
  return (
    <ul>
      {courses.map((c: { id: number; title: string }) => (
        <li key={c.id}>{c.title} <LikeButton /></li>
      ))}
    </ul>
  );
}

// app/courses/LikeButton.tsx   (Client Component)
"use client";
import { useState } from "react";
export default function LikeButton() {
  const [likes, setLikes] = useState(0);
  return <button onClick={() => setLikes(likes + 1)}>Like {likes}</button>;
}
```

Need	Use
Fetch data, read database, keep secrets	Server Component
useState, useEffect, onClick, browser APIs	Client Component
Keep client components small	Push "use client" down to the smallest interactive piece

5.13 Tailwind CSS

Tailwind CSS is a utility-first framework: instead of writing CSS files, you combine small classes directly in HTML or JSX. Installation steps differ between major versions, so follow the official guide for your framework at https://tailwindcss.com/docs/installation.

Tailwind classes

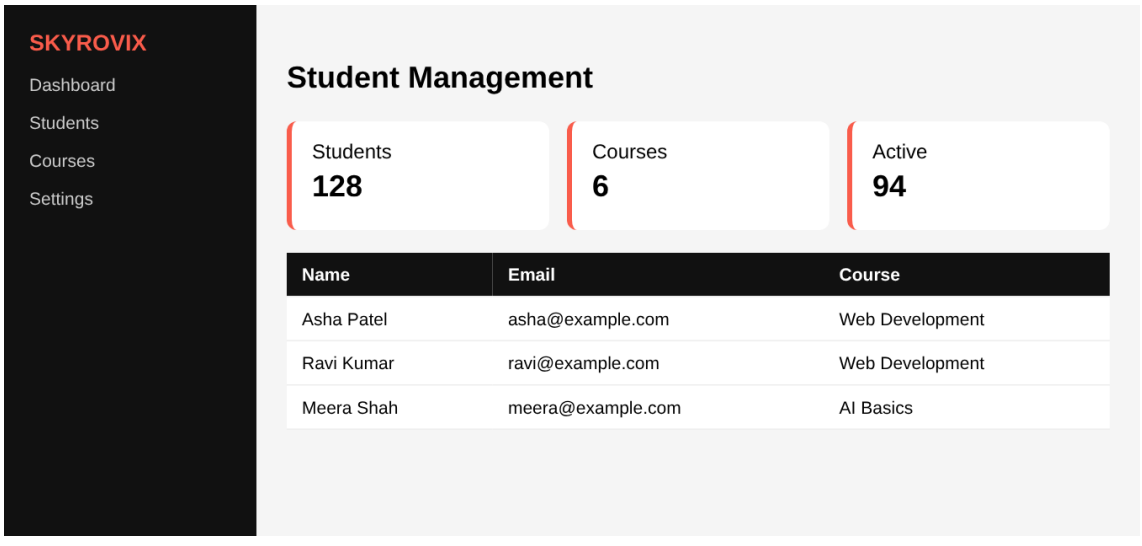
```
export default function Card({ title, text }) {
  return (
    <div className="rounded-xl bg-neutral-100 p-5 shadow-sm hover:shadow-md transition">
      <h3 className="text-lg font-bold text-neutral-900">{title}</h3>
      <p className="mt-2 text-sm text-neutral-600">{text}</p>
      <button className="mt-4 rounded-md bg-[#F95C4B] px-4 py-2 text-white
                         hover:opacity-90 focus-visible:outline">
        Learn more
      </button>
    </div>
  );
}

// responsive grid: 1 column, 2 from md, 3 from lg
<div className="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3"> ... </div>
```

Class names such as `p-5` (padding), `mt-2` (margin top), `text-sm` and `md:grid-cols-2` (applies from the medium breakpoint upward) are explained in the docs. Brand colours are best added to the Tailwind theme configuration per the docs for your version, instead of repeating hex values.

5.14 Frontend Projects

Project 1: Course catalogue with search, filter and detail pages (React Router). **Project 2: Student dashboard** shown below, with a table fed from an API. **Project 3: Next.js blog** with dynamic routes. Build each with at least: loading and error states, a responsive layout and a README.



Screenshot: the Student Dashboard layout you will build in Project 2 (Chromium render of a Grid layout)

Module 5: Common Mistakes and Troubleshooting

- Blank page after a change: open DevTools (F12) Console and read the first red error.
- "Too many re-renders": calling a setter during render. Call it in an event handler or effect.

- Changing state by mutation, so the screen does not update.
- Missing key warnings in lists.
- Importing with the wrong path or file case.
- Hydration errors in Next.js: server and browser HTML differ (e.g. using Date.now() or window during render).

Module 5: Best Practices and Security

- Keep components small and single-purpose.
- Lift state up only as far as needed.
- Never put secrets in frontend or NEXT_PUBLIC_ variables.
- Avoid dangerouslySetInnerHTML with untrusted content.
- Make every interactive element keyboard accessible.
- Run ESLint and test the app on a phone-size screen.

Module 5: Exercises

Beginner exercises

- Create a Counter, then a Todo list, using useState.
- Build a Navbar component that receives links as props.
- Add routes for Home, About and Contact.

Advanced exercises

- Write a useLocalStorage custom hook.
- Build a protected route that redirects when no user is in context.
- Convert a React component to TypeScript with proper prop types.
- Create a Next.js page that fetches data on the server and a client button that updates local state.

Mini project

Build a weather or movie search app: input, results grid, loading and error states, and a detail page using React Router.

Module assignment

Build "SKYROVIX Courses": a responsive React app with login (mock), a protected dashboard, course list from an API, course detail routes, and deployment to Vercel (Module 16). Provide the GitHub link and live URL.

Module 5: Interview Questions

Question	Short model answer
What is the virtual DOM?	A lightweight copy React compares to decide the minimum real DOM updates.
State vs props?	Props are passed in and read-only; state is owned by the component and can change.
Why keys in lists?	They help React track which item is which across renders.
What does useEffect do?	Runs side effects after render; dependencies control re-running.
Controlled component?	A form input whose value is held in React state.
Server vs client components?	Server components render on the server; client components handle interactivity in the browser.
Why TypeScript?	Catches type errors early and improves editor help.

Module 5: Official Documentation and Resources

Resource	Link
React docs	https://react.dev
Vite	https://vite.dev
React Router	https://reactrouter.com
Next.js docs	https://nextjs.org/docs
TypeScript handbook	https://www.typescriptlang.org/docs/
Tailwind CSS	https://tailwindcss.com/docs

MODULE 06

Backend Development

Node.js, Express, REST APIs, authentication, JWT and file uploads

Backend Development

6.1 Backend Development Fundamentals

The **backend** is code that runs on a server, away from the user. It receives **requests**, applies **business rules** (who is allowed to do what), talks to a **database** and sends back **responses**. It is where security matters most, because anything in the browser can be inspected or changed by the user.

Request flow through a typical backend

```
Browser  --HTTP request-->  Router --> Middleware --> Controller --> Database
Browser  <--JSON response--  Router <-- Middleware <-- Controller <--
```

Concept	Meaning
Route	A URL path plus HTTP method, e.g. GET /api/students
Controller	Function that handles a route
Middleware	Function that runs before the controller (logging, auth, validation)
Model	Code that reads and writes data
Environment variables	Settings and secrets kept outside code

6.2 Node.js Complete Guide

Node.js runs JavaScript outside the browser using Chrome's V8 engine. Install the **LTS** version from https://nodejs.org. Node has built-in modules (`fs` for files, `path`, `http`, `crypto`).

hello.js (set "type": "module" in package.json)

```
// hello.js
import { readFile } from "node:fs/promises";
import { createServer } from "node:http";

const text = await readFile("notes.txt", "utf8");
console.log("File has", text.length, "characters");

createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ message: "Hello from Node", url: req.url }));
}).listen(3000, () => console.log("http://localhost:3000"));
```

Run with `node hello.js`. Node is **non-blocking**: while waiting for a file or network, it handles other requests, so use `async/await` and avoid heavy CPU work inside request handlers.

6.3 npm and Packages

npm installs libraries (packages) and records them in `package.json`. The exact versions are locked in `package-lock.json`. Never commit `node_modules`; add it to `.gitignore`.

Common commands

```
npm init -y
npm install express cors dotenv          # dependencies
npm install -D nodemon                   # development-only tool
npm run dev                              # runs the "dev" script
npm audit                                # check known vulnerabilities
npm outdated
```

package.json (scripts)

```
{
  "name": "skyrovix-api",
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

Before installing a package, check weekly downloads, last update, and the repository. Typo-squatted and malicious packages exist, so install only what you need and pay attention to the exact name.

6.4 Express.js

Express (https://expressjs.com) is the most popular Node web framework. It gives you routing and middleware with very little code.

server.js

```
import express from "express";
import cors from "cors";
import "dotenv/config";

const app = express();
app.use(cors({ origin: process.env.CLIENT_URL }));  // allow only your frontend
app.use(express.json());                            // parse JSON bodies

app.get("/", (req, res) => res.send("API is running"));
app.get("/api/students/:id", (req, res) => {
  res.json({ id: req.params.id, query: req.query });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Listening on ${PORT}`));
```

Keep settings in a `.env` file (e.g. PORT=3000, CLIENT_URL=http://localhost:5173) and add `.env` to .gitignore.

6.5 REST API Development

A **REST API** models data as **resources** with URLs, and uses HTTP methods as verbs. Use plural nouns (`/students`), not verbs (`/getStudents`).

routes/students.js (in-memory CRUD)

```
import express from "express";
const router = express.Router();
let students = [{ id: 1, name: "Asha", email: "asha@example.com" }];
let nextId = 2;

router.get("/", (req, res) => res.json(students));

router.get("/:id", (req, res) => {
  const s = students.find(x => x.id === Number(req.params.id));
  if (!s) return res.status(404).json({ error: "Student not found" });
  res.json(s);
});

router.post("/", (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) return res.status(400).json({ error: "name and email are required" });
  const s = { id: nextId++, name, email };
  students.push(s);
  res.status(201).json(s);
});

router.put("/:id", (req, res) => {
  const s = students.find(x => x.id === Number(req.params.id));
  if (!s) return res.status(404).json({ error: "Student not found" });
  Object.assign(s, { name: req.body.name ?? s.name, email: req.body.email ?? s.email });
  res.json(s);
});

router.delete("/:id", (req, res) => {
  students = students.filter(x => x.id !== Number(req.params.id));
  res.status(204).end();
});
export default router;

// in server.js:  app.use("/api/students", router);
```

Data in memory disappears when the server restarts; Module 7 replaces it with a real database. Test with Postman or from the terminal:

Testing with curl

```
curl http://localhost:3000/api/students
curl -X POST http://localhost:3000/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Ravi","email":"ravi@example.com"}'
curl -X DELETE http://localhost:3000/api/students/1
```

6.6 HTTP Methods and Status Codes

Choose the method that matches the intent and return the right status code so clients can react correctly.

Method	Purpose	Idempotent?	Success code
GET	Read	Yes	200
POST	Create	No	201

Method	Purpose	Idempotent?	Success code
PUT	Replace	Yes	200
PATCH	Partially update	Usually	200
DELETE	Remove	Yes	204

Code	Use when
200 OK	Request worked
201 Created	New resource created
204 No Content	Success with nothing to return
400 Bad Request	Invalid input
401 Unauthorized	Not logged in / bad credentials
403 Forbidden	Logged in but not allowed
404 Not Found	Resource does not exist
409 Conflict	e.g. email already registered
422 Unprocessable	Validation failed (also commonly used)
429 Too Many Requests	Rate limit hit
500 Server Error	Unexpected bug

6.7 Middleware

Middleware is a function with access to the request, response and a `next` function. It runs in the order you register it, and each one either ends the response or calls `next()`.

Middleware examples

```
// logger
app.use((req, res, next) => {
  console.log(new Date().toISOString(), req.method, req.url);
  next();
});

// validation middleware
function requireFields(...fields) {
  return (req, res, next) => {
    const missing = fields.filter(f => !req.body?.[f]);
    if (missing.length) return res.status(400).json({ error: "Missing: " + missing.join(", ")
 });
    next();
  };
}
app.post("/api/students", requireFields("name", "email"), createStudent);

// 404 handler: after all routes
app.use((req, res) => res.status(404).json({ error: "Route not found" }));
```

Useful ready-made middleware: **helmet** (safer HTTP headers), **express-rate-limit** (limits requests), **morgan** (logging), **cors**.

6.8 Authentication

Authentication answers "Who are you?". The common flow: the user registers; the password is **hashed**; at login the server checks the password and issues a **token** or session; the browser sends it with later requests. You can build this yourself to learn, but in production consider a proven service such as Supabase Auth or Firebase Auth (Modules 13 and 14), because authentication is easy to get subtly wrong.

Auth flow

```
Register: email + password -> hash password -> save user
Login:    email + password -> compare hash -> issue JWT
Request:  Authorization: Bearer <token> -> verify -> req.user -> controller
```

6.9 Authorization

Authorization answers "What are you allowed to do?". After you know who the user is, check their **role** or **ownership** of the data. Checking happens on the server for every request; hiding a button in the frontend is not security.

Role-based access control

```
function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ error: "Forbidden" });
    }
    next();
  };
}
app.delete("/api/students/:id", requireAuth, requireRole("admin"), removeStudent);

// ownership check
if (note.userId !== req.user.id) return res.status(403).json({ error: "Not your note" });
```

6.10 JWT

A **JSON Web Token** is a signed string containing claims such as the user id. It has three parts (header, payload, signature) separated by dots. The payload is **encoded, not encrypted**: anyone can read it, so never put secrets in it. The signature proves it was not changed. Library: `jsonwebtoken`.

auth middleware

```
import jwt from "jsonwebtoken";

function signToken(user) {
  return jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: "1h
" });
}

export function requireAuth(req, res, next) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: "Login required" });
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = { id: payload.sub, role: payload.role };
    next();
  } catch {
    res.status(401).json({ error: "Invalid or expired token" });
  }
}
```

Keep the secret long and random and only in environment variables. Use short expiry times. Where you store the token in the browser matters: tokens in localStorage can be stolen by XSS; **httpOnly cookies** cannot be read by JavaScript but need CSRF protection. Read the OWASP guidance at https://owasp.org before choosing.

6.11 Password Hashing

Never store passwords as text. A **hash** is a one-way scrambled value; with a good algorithm and a random **salt**, even you cannot recover the original password. Use a slow, purpose-built algorithm such as **bcrypt** or **argon2**, never plain MD5 or SHA-256.

Register and login

```
import bcrypt from "bcrypt";

app.post("/api/auth/register", async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password || password.length < 8)
    return res.status(400).json({ error: "Email and 8+ character password required" });
  const hash = await bcrypt.hash(password, 12);       // 12 = cost factor
  const user = await db.createUser({ email, passwordHash: hash });
  res.status(201).json({ id: user.id, email: user.email });  // never return the hash
});

app.post("/api/auth/login", async (req, res) => {
  const user = await db.findUserByEmail(req.body.email);
  const ok = user && await bcrypt.compare(req.body.password, user.passwordHash);
  if (!ok) return res.status(401).json({ error: "Wrong email or password" });  // same messag
e for both
  res.json({ token: signToken(user) });
});
```

6.12 File Uploads

Browsers send files as `multipart/form-data`. In Express, the **multer** package handles it. Always limit size and type, rename files, and prefer cloud storage (Supabase Storage, Firebase Storage, S3) over your server's disk in production.

Safe image upload

```
import multer from "multer";
import path from "node:path";
import crypto from "node:crypto";

const storage = multer.diskStorage({
  destination: "uploads/",
  filename: (req, file, cb) =>
    cb(null, crypto.randomUUID() + path.extname(file.originalname).toLowerCase()),
});
const upload = multer({
  storage,
  limits: { fileSize: 2 * 1024 * 1024 },                    // 2 MB
  fileFilter: (req, file, cb) =>
    cb(null, ["image/png", "image/jpeg"].includes(file.mimetype)),
});

app.post("/api/avatar", requireAuth, upload.single("avatar"), (req, res) => {
  if (!req.file) return res.status(400).json({ error: "PNG or JPEG required" });
  res.status(201).json({ file: req.file.filename });
});
```

The browser form needs `enctype="multipart/form-data"`, or use `FormData` with fetch. Do not trust the file name or the type sent by the client alone.

6.13 Error Handling

Handle errors in one place so every failure returns a clean response and no internal details leak.

Central error handler

```
// async wrapper so errors reach the handler (needed in Express 4)
const wrap = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

app.get("/api/students/:id", wrap(async (req, res) => {
  const s = await db.getStudent(req.params.id);
  if (!s) throw Object.assign(new Error("Student not found"), { status: 404 });
  res.json(s);
}));

// error middleware has FOUR parameters and goes last
app.use((err, req, res, next) => {
  console.error(err);                                  // log full details on the server
  const status = err.status || 500;
  res.status(status).json({ error: status === 500 ? "Something went wrong" : err.message });
});
```

Also handle `process.on('unhandledRejection')` in production and use a process manager or the host's restart policy.

6.14 Backend Projects

Project 1: Notes API with register, login and notes that belong to the logged-in user. **Project 2: Student API** with roles (admin can delete, others read). **Project 3: Image gallery** with uploads. For each: write a README with endpoint tables, test every endpoint in Postman, and add rate limiting and helmet.

Module 6: Common Mistakes and Troubleshooting

- EADDRINUSE: the port is already used. Stop the other process or change PORT.
- "Cannot POST /route": method or path mismatch; check app.use prefixes.
- req.body is undefined: missing express.json() before routes.
- CORS errors in the browser: configure the allowed origin on the server; the error is shown in the browser but must be fixed on the backend.
- Crashes on async errors: use try/catch or the wrapper in 6.13.
- Forgetting to restart the server after editing code (use nodemon).

Module 6: Best Practices and Security

- Hash passwords with bcrypt or argon2; never log passwords or tokens.
- Validate and sanitise all input (libraries like zod or joi help).
- Use parameterised queries (Module 7) to prevent SQL injection.
- Use HTTPS, helmet, rate limiting, and restrict CORS.
- Return the same error for wrong email and wrong password.
- Keep dependencies updated and run npm audit.
- Follow OWASP Top 10 (https://owasp.org/www-project-top-ten/).

Module 6: Exercises

Beginner exercises

- Create a server with three GET routes returning JSON.
- Add a logging middleware.
- Build full CRUD for a "books" resource in memory and test with curl.

Advanced exercises

- Add validation using zod to POST and PUT.
- Implement register, login and a protected /me route with JWT.
- Add pagination (?page=2&limit;=10) and search (?q=) to a list endpoint.
- Add rate limiting to the login route.

Mini project

Build a Notes API: users register and login, create, list, update and delete only their own notes.

Module assignment

Build the backend for the Student Management System: authentication, roles (admin/teacher), CRUD routes, validation, error handling, file upload for profile photos, and API documentation in the README.

Module 6: Interview Questions

Question	Short model answer
What is middleware?	A function in the request pipeline that can modify the request or response or stop the flow.
PUT vs PATCH?	PUT replaces a resource; PATCH changes part of it.
Authentication vs authorization?	Who you are vs what you may do.
Why hash passwords?	So a database leak does not reveal them; hashes are one way.
What is in a JWT?	Header, payload (claims) and signature; readable but tamper-evident.
What is CORS?	A browser security rule; servers state which origins may call them.
What does idempotent mean?	Repeating the request gives the same result as doing it once.

Module 6: Official Documentation and Resources

Resource	Link
Node.js docs	https://nodejs.org/docs/latest/api/
Express	https://expressjs.com
MDN HTTP	https://developer.mozilla.org/en-US/docs/Web/HTTP
OWASP Top 10	https://owasp.org/www-project-top-ten/
jwt.io introduction	https://jwt.io/introduction

MODULE 07

Database Management

SQL, MySQL, PostgreSQL, design, relationships, security

Database Management

7.1 Database Fundamentals

A **database** is organised, permanent storage for data. A **DBMS** (database management system) such as PostgreSQL or MySQL manages it. A **relational database** stores data in **tables** (rows are records, columns are fields) and connects tables using keys. **ACID** properties (atomicity, consistency, isolation, durability) mean a **transaction**, such as transferring money, either fully happens or does not happen at all.

Term	Meaning
Table	A set of rows with the same columns
Row / record	One item (one student)
Column / field	One property (email)
Schema	The structure: tables, columns, types, rules
Query	A request to read or change data
Index	A lookup structure that makes searches faster

7.2 SQL Fundamentals

SQL (Structured Query Language) is the language for relational databases. The examples below ran for real in SQLite while this book was built; the same statements work in PostgreSQL and MySQL with small differences.

Create and fill tables

```
CREATE TABLE courses (
  id INTEGER PRIMARY KEY, title TEXT NOT NULL, fee INTEGER);
CREATE TABLE students (
  id INTEGER PRIMARY KEY, name TEXT NOT NULL,
  email TEXT UNIQUE, course_id INTEGER REFERENCES courses(id));
INSERT INTO courses VALUES (1,'Web Development',9000),
  (2,'AI Basics',7000),(3,'UI Design',6000);
INSERT INTO students VALUES
  (1,'Asha','asha@example.com',1),(2,'Ravi','ravi@example.com',1),
  (3,'Meera','meera@example.com',2),(4,'Kabir','kabir@example.com',NULL);
```

Query

```
SELECT name, email FROM students WHERE course_id = 1 ORDER BY name;
```

Real output

```
name  email
----  ----------------
Asha  asha@example.com
Ravi  ravi@example.com
```

Query

```
SELECT title, fee FROM courses WHERE fee >= 7000 ORDER BY fee DESC;
```

Real output

```
title           fee
--------------  ----
Web Development  9000
AI Basics        7000
```

Query

```
SELECT COUNT(*) AS total_students FROM students;
```

Real output

```
total_students
--------------
4
```

Core statements: **SELECT** (read), **INSERT** (create), **UPDATE** (change), **DELETE** (remove). Always use `WHERE` with UPDATE and DELETE, or every row is affected.

More examples

```
UPDATE students SET course_id = 2 WHERE id = 4;
DELETE FROM students WHERE email = 'ravi@example.com';
SELECT * FROM students WHERE name LIKE 'A%' AND course_id IS NOT NULL;
SELECT course_id, COUNT(*) FROM students GROUP BY course_id HAVING COUNT(*) > 1;
```

7.3 MySQL

MySQL (https://dev.mysql.com/doc/) is a very popular open-source relational database, widely used with PHP and on shared hosting. Install MySQL Community Server or use a managed service. Notable differences: `AUTO_INCREMENT` for ids, backticks around names, `LIMIT` for paging, and the `InnoDB` engine for transactions and foreign keys.

MySQL syntax

```
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Using MySQL from Node

```
// Node.js with mysql2: use ? placeholders (parameterised)
import mysql from "mysql2/promise";
const pool = mysql.createPool({ host: process.env.DB_HOST, user: process.env.DB_USER,
  password: process.env.DB_PASS, database: process.env.DB_NAME });
const [rows] = await pool.query("SELECT * FROM students WHERE email = ?", [email]);
```

7.4 PostgreSQL

PostgreSQL (https://www.postgresql.org/docs/) is a powerful open-source database known for standards compliance, JSON support and advanced features. It is the database behind Supabase (Module 13). Notable syntax: `SERIAL` or identity columns for ids, `TEXT`, `TIMESTAMPTZ`, `JSONB`, and `$1, $2` placeholders.

PostgreSQL syntax

```
CREATE TABLE students (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  profile JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Using PostgreSQL from Node

```
// Node.js with pg
import pg from "pg";
const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });
const { rows } = await pool.query("SELECT * FROM students WHERE email = $1", [email]);
```

	MySQL	PostgreSQL
Strengths	Simple, very common, good read speed	Rich features, standards, JSONB, extensions
Typical use	WordPress, PHP apps, shared hosting	Modern apps, Supabase, analytics
Auto id	AUTO_INCREMENT	IDENTITY / SERIAL
Placeholders	?	$1, $2

7.5 Database Design

Good design avoids duplicated data. **Normalisation** means organising tables so each fact is stored once. Steps: 1) one value per cell (no lists in a column); 2) every column depends on the whole key; 3) columns depend on the key and nothing else. Design process: list the **entities** (things), their **attributes**, then the **relationships**.

Bad design	Better design
students(name, courses="HTML,CSS,JS")	students + courses + enrollments (one row per enrollment)
orders(customer_name, customer_phone...) repeated per order	customers table and orders.customer_id
Storing age	Store date_of_birth; calculate age

7.6 Tables and Relationships

Relationships connect tables.

Type	Example	How
One-to-one	user and profile	Foreign key with UNIQUE
One-to-many	one course has many students	Foreign key on the "many" side
Many-to-many	students and courses	Junction table with two foreign keys

Junction table for many-to-many

```
CREATE TABLE enrollments (
  student_id INTEGER REFERENCES students(id) ON DELETE CASCADE,
  course_id  INTEGER REFERENCES courses(id)  ON DELETE CASCADE,
  enrolled_on DATE DEFAULT CURRENT_DATE,
  PRIMARY KEY (student_id, course_id)
);
```

`ON DELETE CASCADE` deletes dependent rows automatically; `RESTRICT` blocks deletion. Choose deliberately.

7.7 Primary and Foreign Keys

A **primary key** uniquely identifies each row; it cannot be null or duplicated. A **foreign key** is a column whose values must exist in another table's primary key, so you cannot enrol a student in a course that does not exist. Other constraints: `NOT NULL`, `UNIQUE`, `CHECK` (e.g. `CHECK (fee >= 0)`), `DEFAULT`. Use surrogate ids (numbers or UUIDs) rather than emails as primary keys, since emails can change.

7.8 CRUD Operations

CRUD stands for Create, Read, Update, Delete: the four basic data actions. They map to SQL and to REST like this:

CRUD	SQL	HTTP
Create	INSERT	POST
Read	SELECT	GET
Update	UPDATE	PUT / PATCH
Delete	DELETE	DELETE

CRUD with PostgreSQL

```
// Express + pg: Create and Read with parameterised queries
router.post("/", async (req, res) => {
  const { name, email, course_id } = req.body;
  const { rows } = await pool.query(
    "INSERT INTO students (name, email, course_id) VALUES ($1, $2, $3) RETURNING *",
    [name, email, course_id]);
  res.status(201).json(rows[0]);
});
router.get("/", async (req, res) => {
  const limit = Math.min(Number(req.query.limit) || 20, 100);
  const offset = Number(req.query.offset) || 0;
  const { rows } = await pool.query("SELECT * FROM students ORDER BY id LIMIT $1 OFFSET $2",
[limit, offset]);
  res.json(rows);
});
```

7.9 Joins and Queries

A **JOIN** combines rows from two tables using a related column. **INNER JOIN** keeps only matching rows; **LEFT JOIN** keeps all rows from the left table, with empty values where nothing matches. The results below were produced by running the queries.

INNER JOIN

```
SELECT s.name, c.title FROM students s INNER JOIN courses c ON c.id = s.course_id ORDER BY s.
name;
```

Real output

```
name   title
-----  ---------------
Asha   Web Development
Meera  AI Basics
Ravi   Web Development
```

LEFT JOIN: Kabir has no course

```
SELECT s.name, c.title FROM students s LEFT JOIN courses c ON c.id = s.course_id ORDER BY s.n
ame;
```

Real output

```
name   title
-----  ---------------
Asha   Web Development
Kabir  None
Meera  AI Basics
Ravi   Web Development
```

Aggregates with GROUP BY

```
SELECT c.title, COUNT(s.id) AS students, SUM(c.fee) AS fee_total FROM courses c LEFT JOIN stu
dents s ON s.course_id = c.id GROUP BY c.id ORDER BY students DESC;
```

Real output

```
title           students  fee_total
--------------  --------  ---------
Web Development  2         18000
AI Basics        1         7000
UI Design        0         6000
```

Subquery, index and EXPLAIN

```
-- Subquery and index
SELECT name FROM students WHERE course_id IN (SELECT id FROM courses WHERE fee > 6500);
CREATE INDEX idx_students_course ON students(course_id);
EXPLAIN SELECT * FROM students WHERE course_id = 1;   -- see how the database runs it
```

Indexes speed up reads but slow writes and use space; index columns you filter, join or sort on often. Avoid SELECT * in production code; list the columns you need.

7.10 Supabase

Supabase gives you managed PostgreSQL with authentication, storage and automatic APIs. All of Module 13 is dedicated to it. In relation to this module: your SQL knowledge applies directly, the SQL Editor runs the statements above, and Row Level Security policies replace much backend authorisation code.

7.11 Firebase

Firebase Firestore is a NoSQL document database: data lives in collections of documents rather than tables. Joins do not exist, so you duplicate or restructure data for the screens you need. See Module 14. Choose SQL when data is highly related and consistency matters; choose Firestore for quick real-time apps with flexible data.

7.12 Database Authentication

Two different things are called authentication here. (1) **App users** logging in, handled by your backend or a service such as Supabase Auth. (2) **Database accounts** your application uses to connect. Give the application its own database user with only the permissions it needs, never the superuser.

Least privilege

```
-- PostgreSQL: least-privilege application user
CREATE ROLE app_user LOGIN PASSWORD 'use-a-long-random-password';
GRANT CONNECT ON DATABASE school TO app_user;
GRANT SELECT, INSERT, UPDATE ON students TO app_user;   -- no DELETE, no DROP
```

7.13 Database Security

- **SQL injection:** attackers type SQL into form fields. Prevent it by always using parameterised queries, never string concatenation.
- **Least privilege:** limited database users; separate read and write roles where useful.
- **Secrets:** connection strings only in environment variables; rotate if leaked.
- **Encryption:** use TLS connections; hash passwords; encrypt very sensitive fields.
- **Backups:** automate and test restoring them. A backup you never restored is a hope, not a backup.

- **Row Level Security** (PostgreSQL / Supabase) restricts which rows a user can access.
- **Do not expose** the database port to the public internet.

SQL injection: wrong vs right

```
// NEVER do this: attacker can send  ' OR '1'='1
const q = "SELECT * FROM students WHERE email = '" + email + "'";

// DO this: the value is sent separately from the SQL text
const { rows } = await pool.query("SELECT * FROM students WHERE email = $1", [email]);
```

7.14 Practical Database Projects

Project 1: Library system. Tables: books, members, loans. Queries: overdue loans, most borrowed books. **Project 2: Student Management database** with students, courses, enrollments and marks; write queries for average marks per course and students with no enrollment. **Project 3: E-commerce schema** with customers, products, orders and order_items; calculate order totals with joins. Draw an ER diagram (https://dbdiagram.io is a free option; check its current terms) before writing SQL.

Module 7: Common Mistakes and Troubleshooting

- "Violates foreign key constraint": inserting a reference to a row that does not exist, or deleting a row that is still referenced.
- "Duplicate key": inserting a value that must be unique.
- Forgetting WHERE in UPDATE or DELETE. Run a SELECT with the same WHERE first.
- Wrong join type causing missing rows (INNER instead of LEFT).
- Using = NULL instead of IS NULL.
- Storing numbers or dates as text.

Module 7: Best Practices and Security

- Use parameterised queries everywhere.
- Design with keys and constraints so the database protects your data.
- Use transactions for multi-step changes.
- Index carefully and measure with EXPLAIN.
- Keep migrations (schema changes) in version control.
- Use separate databases for development and production; test restores.

Module 7: Exercises

Beginner exercises

- Create a table of 5 books and query by author.
- Update one record and delete another safely.
- Write a join listing each student with their course.

Advanced exercises

- Design an enrollment system with a junction table and write a query that lists courses with their student counts, including courses with none.
- Add an index and compare EXPLAIN output before and after.

- Write a transaction that creates an order and its items together.

Mini project

Create the Library database with books, members and loans, and write 10 useful queries.

Module assignment

Design and implement the database for the Student Management System: ER diagram, SQL schema with constraints, sample data, 12 queries (including joins and aggregates), and a security checklist.

Module 7: Interview Questions

Question	Short model answer
SQL vs NoSQL?	SQL uses structured tables with relationships; NoSQL uses flexible models such as documents.
Primary vs foreign key?	Primary identifies a row; foreign links to another table.
INNER vs LEFT JOIN?	INNER returns matches only; LEFT keeps all left rows.
What is normalisation?	Organising data to avoid duplication and anomalies.
What is an index?	A structure that speeds up lookups at some cost to writes.
What is SQL injection?	Injecting SQL through input; prevented with parameterised queries.
What is a transaction?	A group of operations that succeed or fail together.

Module 7: Official Documentation and Resources

Resource	Link
PostgreSQL docs	https://www.postgresql.org/docs/
MySQL docs	https://dev.mysql.com/doc/
SQLite docs	https://www.sqlite.org/docs.html
PostgreSQL tutorial site	https://www.postgresqltutorial.com
OWASP SQL injection	https://owasp.org/www-community/attacks/SQL_Injection

MODULE 08

AI Tools for Web Developers

ChatGPT, Claude, Gemini, Antigravity, Cursor, Copilot, Lovable, Bolt.new, v0, Replit

AI Tools for Web Developers

AI coding tools predict useful code from your description. They are fast, but they make mistakes and can invent things that do not exist. **You remain responsible** for understanding, testing and securing what they produce. Two categories matter:

- **Chat assistants** (ChatGPT, Claude, Gemini): you ask in a chat window and copy results.
- **Coding environments and app builders** (Antigravity, Cursor, Copilot, Replit): work inside your project files. **Lovable, Bolt.new and v0** generate whole apps from prompts in the browser.

Pricing and limits
Not printed here on purpose: plans, quotas and prices for AI tools change every few months. Free tiers exist for many tools, but check the official pricing page on the day you sign up and note the date.

ChatGPT

Item	Details
What is it?	Chat assistant for explaining, writing and debugging code.
Made by	OpenAI
Official site	https://chatgpt.com
Account and setup	Sign up with email, Google or other listed options at the site. Also available as mobile and desktop apps.
Use for web development	Paste an error message and ask what it means; ask for a CSS layout; ask it to explain a function line by line.
Advantages and limits	Fast explanations. Cannot see your project unless you paste it; can sound confident and be wrong.
Best use cases	Learning, debugging snippets, planning.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Claude

Item	Details
What is it?	Chat assistant strong at long code, reasoning and writing. Claude Code is a separate coding tool.
Made by	Anthropic
Official site	https://claude.ai
Account and setup	Sign up at claude.ai with email or Google. Web, mobile and desktop apps exist.
Use for web development	Upload a file or paste code for review; ask for a refactor with explanations; request test cases.
Advantages and limits	Handles long inputs well. Still needs checking; you copy results manually unless you use a coding tool.
Best use cases	Code review, long files, documentation.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Google Gemini

Item	Details
What is it?	Google's AI assistant, tied into Google accounts and services.
Made by	Google
Official site	https://gemini.google.com
Account and setup	Sign in with a Google account.
Use for web development	Ask for HTML/CSS/JS examples, explanations of errors, or help understanding Google APIs.
Advantages and limits	Easy if you already use Google. Same accuracy caveats as other chat assistants.
Best use cases	Quick help, Google ecosystem tasks.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Google Antigravity

Item	Details
What is it?	Agent-first desktop development environment (see Part 04).
Made by	Google
Official site	https://antigravity.google
Account and setup	Download from the official site and sign in with a Google account.
Use for web development	Describe a feature; an agent plans, edits files and can test in a browser.
Advantages and limits	Powerful agents; quotas are tied to Google AI plans and have changed repeatedly, so check current limits.
Best use cases	Building features with agents, prototypes.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Cursor

Item	Details
What is it?	Code editor built around AI, similar in feel to VS Code (see Part 05).
Made by	Anysphere
Official site	https://cursor.com
Account and setup	Download from cursor.com and create an account.
Use for web development	Ask questions about your codebase; let the agent edit several files; fix bugs from error output.
Advantages and limits	Understands whole projects. Pricing structure changed in late 2026, so verify the current plans.
Best use cases	Daily coding on real projects.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

GitHub Copilot

Item	Details
What is it?	AI assistant inside VS Code and other editors: suggestions, chat and agent features.
Made by	GitHub / Microsoft
Official site	https://github.com/features/copilot
Account and setup	Needs a GitHub account; enable Copilot from your GitHub settings and install the extension in VS Code.
Use for web development	Accept inline suggestions while typing; open Copilot Chat to explain or fix selected code.
Advantages and limits	Works inside your editor. Students may qualify for free access via GitHub Education; check eligibility.
Best use cases	Everyday suggestions in VS Code.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Lovable

Item	Details
What is it?	Browser-based builder that generates full web apps from prompts.
Made by	Lovable
Official site	https://lovable.dev
Account and setup	Sign up on the website.
Use for web development	Prompt: "Build a student portfolio with a contact form" then refine by chatting.
Advantages and limits	Very fast first version. Generated code quality and limits vary; always review before launching.
Best use cases	Prototypes, landing pages.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Bolt.new

Item	Details
What is it?	Browser-based AI app builder that creates and runs projects online.
Made by	StackBlitz
Official site	https://bolt.new
Account and setup	Sign up on the website.
Use for web development	Describe an app; it creates the files and a live preview you can edit by chat.
Advantages and limits	No local setup needed. Complex apps can drift; check usage limits on the site.
Best use cases	Quick full-stack prototypes.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

v0

Item	Details
What is it?	AI tool that generates UI components and apps, closely linked to React and Vercel.
Made by	Vercel
Official site	https://v0.app
Account and setup	Sign up with a Vercel or supported login at the site.
Use for web development	Prompt: "A pricing page with three cards" and copy the React code into your project.
Advantages and limits	Good-looking UI fast. Best if you already use React.
Best use cases	UI components and layouts.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Replit AI

Item	Details
What is it?	Online coding workspace with built-in AI and hosting options.
Made by	Replit
Official site	https://replit.com
Account and setup	Sign up at replit.com.
Use for web development	Create a project in the browser, ask the AI agent to build or fix features, and run it online.
Advantages and limits	Works from any device. Limits and features depend on plan; verify on the site.
Best use cases	Learning, sharing runnable projects.
Free / paid	See the official pricing page. Verify the free allowance and paid plans on the date you sign up.

Comparison table

Tool	Type	Works in your files?	Good for beginners?
ChatGPT	Chat	No (copy/paste)	Yes
Claude	Chat	No (copy/paste)	Yes
Gemini	Chat	No (copy/paste)	Yes
Antigravity	Agent IDE	Yes	Yes, with care
Cursor	AI editor	Yes	Yes
Copilot	Editor extension	Yes	Yes
Lovable	App builder	Generated project	Yes for prototypes
Bolt.new	App builder	Generated project	Yes for prototypes
v0	UI generator	Copy into project	If you know React basics
Replit AI	Online IDE	Yes (in browser)	Yes

How to write good prompts

- State the goal, the tech (HTML/CSS/React) and the constraints: "responsive, no external libraries".
- Give an example of input and expected output.
- Ask for small steps, not the whole app at once.
- Paste the exact error message and the code that caused it.
- Ask: "explain this code so a beginner understands".

> **Rules for using AI responsibly**
> 1) Never paste passwords, API keys or private data. 2) Run and test everything. 3) Read the code until you understand it, or you will not be able to fix it. 4) Follow your college or employer policy on AI use.

Exercise
Ask two different assistants for the same task: "Create a responsive navigation bar with HTML and CSS". Compare the answers, find one thing in each you would improve, and fix it by hand.

MODULE 09

Visual Studio Code Complete Guide

Install, configure and use the world's most popular free code editor

Visual Studio Code Complete Guide

Visual Studio Code (VS Code) is a free code editor from Microsoft that runs on Windows, macOS and Linux. It is lightweight but can be extended with **extensions** to work like a full development environment. Official site: https://code.visualstudio.com

Install

- Open https://code.visualstudio.com and download the installer for your operating system.
- Windows: run the installer and tick **Add to PATH** and **Add "Open with Code"** options if offered.
- macOS: unzip and drag Visual Studio Code to Applications.
- Open VS Code and choose File > Open Folder to start working inside a project folder.

Interface tour

Area	What it does
Activity Bar (left edge)	Switch between Explorer, Search, Source Control, Run and Debug, Extensions.
Side Bar	Shows the panel for the selected activity, e.g. your files.
Editor	Where you write code. Open several files in tabs or split them.
Panel (bottom)	Terminal, Problems, Output and Debug Console.
Status Bar	Branch name, errors, language, line and column.
Command Palette	Press Ctrl+Shift+P (Cmd+Shift+P on Mac) to run any command by name.

Useful extensions

Extension	Why
Live Server	Opens a local server that reloads the browser when you save.
Prettier	Formats code automatically.
ESLint	Highlights JavaScript problems.
GitHub Copilot	AI suggestions (needs an account).
GitLens	Shows who changed each line in Git.

Install from the Extensions view (Ctrl+Shift+X). Check the publisher name and install count before installing; fake extensions exist.

HTML, CSS and JavaScript workflow

- Create a folder skyrovix-site and open it in VS Code.
- Create index.html and type ! then press Tab: Emmet expands a full HTML skeleton.
- Create styles.css and script.js and link them.
- Right-click index.html and choose "Open with Live Server" if you installed it.

Create a React project

Install Node.js first (see Part 01). In the VS Code terminal (Ctrl+`) run:

Terminal

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Open the address shown (usually http://localhost:5173). Edit src/App.jsx and the page updates when you save.

Git and GitHub in VS Code

- Open the Source Control view. Click **Initialize Repository** (or use `git init`).
- Type a message and click **Commit**.
- Use **Publish Branch** to push to GitHub; sign in when asked.

Debugging

- Click left of a line number to set a **breakpoint** (red dot).
- Open Run and Debug and start a configuration (for Node.js choose Node.js).
- Use Step Over, Step Into and Continue; inspect variables in the Variables panel.
- In the browser, press F12 for Chrome DevTools.

Keyboard shortcuts (Windows/Linux; use Cmd for Ctrl on Mac)

Shortcut	Action
Ctrl+P	Open a file by name
Ctrl+Shift+P	Command Palette
Ctrl+S	Save
Ctrl+/	Comment or uncomment
Alt+Up / Down	Move line
Ctrl+D	Select next match
Ctrl+`	Toggle terminal
Ctrl+B	Toggle side bar
Ctrl+F / Ctrl+H	Find / Replace

Practical project
Build a 2-page portfolio in VS Code, use Live Server to preview, commit with Git, and publish it to GitHub. Take screenshots of your own screen for your records.

Common mistakes
Opening a single file instead of the project folder; editing without saving; not trusting the Problems panel.

MODULE 10

Google Antigravity Complete Guide

An agent-first development environment from Google

Google Antigravity Complete Guide

Google Antigravity is a Google development environment built around **AI agents**: programs that plan a task, edit files, run commands and can check work in a browser. It launched in public preview in November 2025 and has been updated since (Google announced a version 2.0 at I/O 2026). Official site: https://antigravity.google

Verify before you follow
Antigravity changes quickly. Menu names and plan limits below are described in general terms; confirm against the official documentation linked on its website. Plans: reports say Antigravity has no separate subscription; paid usage comes through Google AI plans, with a free individual tier whose limits have been reduced over time.

Install and first setup

- Download the installer for your system from the official Antigravity site.
- Run it, open Antigravity, and sign in with your Google account.
- Choose your preferences when prompted, then open a project folder.

Understanding agents

Idea	Meaning
Agent	An AI worker that can plan steps, change files and run tools for a task.
Agent Manager	The area for starting and watching one or more agents working on tasks.
Review step	You inspect changes before accepting them. Always do this.
Quota	How much agent work your plan allows over time.

Build a website with an agent

- Create an empty folder named skyrovix-portfolio and open it.
- Give a clear prompt (below).
- Read the plan the agent proposes; correct it if needed.
- Run the project and test in the browser; ask the agent to fix specific problems.

Example prompt

```
Build a responsive portfolio website using plain HTML, CSS and JavaScript.
Pages: Home, Projects, Contact.
Brand colours: #F95C4B and #111111. No external libraries.
Contact form must validate name, email and message.
Explain the folder structure when you finish.
```

Writing effective prompts

- One task per prompt; list pages, features and technology.
- Name files you want changed.
- State what must NOT change.
- Ask for explanations so you learn.

Best practices

- Use Git so you can undo agent changes.
- Never give agents secrets such as API keys in prompts.
- Review every diff; run the app after each task.
- Watch your quota; start with small tasks.

Project
Use Antigravity to build the portfolio above, then rebuild one page by hand without AI. Compare the code and note what you learned.

MODULE 11

Cursor AI Complete Guide

An AI-first code editor for real projects

Cursor AI Complete Guide

Cursor is a code editor built for AI-assisted programming. It looks and feels like VS Code, so your shortcuts mostly carry over. Official site: https://cursor.com

Plans
Cursor has a free Hobby plan and paid plans. In October 2026 Cursor introduced a new set of plans, and OpenAI announced it would stop supplying models to Cursor from a proposed date of 12 November 2026. Always read https://cursor.com/pricing before choosing.

Install and setup

- Download from cursor.com and install.
- Create an account and sign in.
- Optionally import your VS Code settings and extensions when asked.
- Open a project folder.

Main features

Feature	What it does	Typical use
Tab completion	Suggests the next edit as you type	Everyday coding
Chat	Ask questions about your code	"Where is login handled?"
Codebase context	Lets the AI search your project	Understand unfamiliar code
Agent / Composer-style editing	Makes multi-file changes from an instruction	Add a feature, then review the diff
Rules files	Project instructions the AI follows	Code style, tech choices

Workflows

- **Understand:** ask "Explain the folder structure and how data flows".
- **Generate:** describe a feature, keep it small.
- **Fix bugs:** paste the error and the file; ask for the cause before the fix.
- **Refactor:** "Split this 300-line component into smaller ones without changing behaviour".
- **Existing projects:** open the folder, ask for a summary first, then work in small steps with Git commits.

Example agent prompt

```
Add a "Delete student" button to StudentList.jsx.
It must call DELETE /api/students/:id, then refresh the list.
Show an error message if the request fails.
Do not change any other file.
```

Practical full-stack project
Build a Student Management app: React frontend, Express API, and a database (Part 07 or 08). Use Cursor for each feature, commit after each step, and write 5 test cases by hand.

Common mistakes
Accepting large changes without reading; vague prompts; leaving secrets inside code; ignoring errors the AI introduces.

MODULE 12

Figma UI/UX Design Guide

Design a website before you code it

Figma UI/UX Design Guide

Figma is a browser-based design tool for interfaces. **UI** (user interface) is how a product looks; **UX** (user experience) is how it feels to use. Official site: https://www.figma.com. Figma has a free plan; check the site for current plan details.

Getting started

- Go to figma.com and sign up with email or Google.
- Create a new Design file.
- Use the toolbar to draw frames, shapes, text and images.

Core concepts

Concept	Explanation
Frame	A container like a screen or section. Pick a preset such as Desktop or Phone.
Layer	Every object appears in the layers list; name them clearly.
Typography	Choose 1-2 fonts, set sizes for headings and body, keep line height comfortable.
Colour styles	Save brand colours (#F95C4B, #111111, #F5F5F5, #FFFFFF) as styles.
Component	A reusable element such as a button. Change the main one and all copies update.
Variant	Different states of one component (default, hover, disabled).
Auto Layout	Makes frames resize and space items automatically, like CSS Flexbox.
Prototype	Connect frames with clickable links to simulate the real flow.

Design workflow

- **1. Wireframe:** grey boxes only. Decide layout and content before colour.
- **2. Style:** add typography, colours, spacing.
- **3. Components:** turn buttons, cards, nav into components.
- **4. Responsive:** design desktop, tablet and mobile frames; use Auto Layout.
- **5. Prototype:** link pages and test with a friend.
- **6. Handoff:** share the file; developers inspect spacing, colours and export assets.

Design system basics

Element	Example rule
Spacing	Use multiples of 8 (8, 16, 24, 32).
Type scale	H1 40, H2 28, H3 20, body 16.
Colour	Primary, text, background, error.
Components	Button, input, card, navbar.

Project
Design a 3-screen website for a fictional coaching class: Home, Courses, Contact. Create a button component with two variants, use Auto Layout for the course cards, link the screens in a prototype, then build it in HTML/CSS (Part 01).

MODULE 13

Supabase Complete Guide

A PostgreSQL backend with authentication and storage

Supabase Complete Guide

Supabase is a backend platform built on **PostgreSQL** (a popular SQL database). It gives you a database, user authentication, file storage and an automatic API. Official site: https://supabase.com and docs at https://supabase.com/docs. A free plan exists; check current limits on the pricing page.

Create a project

- Sign up at supabase.com (GitHub login is common).
- Click New project, choose an organisation, name it, set a strong database password and pick a region.
- Wait until the project finishes setting up.
- Find your Project URL and anon public key under the project API settings. The anon key is meant for browsers; never put a **service role** key in frontend code.

Tables and relationships

SQL Editor

```
create table courses (
  id bigint generated always as identity primary key,
  title text not null
);

create table students (
  id bigint generated always as identity primary key,
  name text not null,
  email text unique,
  course_id bigint references courses(id),
  created_at timestamptz default now()
);

insert into courses (title) values ('Web Development');
insert into students (name, email, course_id)
values ('Asha', 'asha@example.com', 1);

select s.name, c.title
from students s join courses c on c.id = s.course_id;
```

A **primary key** uniquely identifies a row. A **foreign key** (references) links a row to another table: each student belongs to one course.

Authentication

Enable the Email provider in Authentication settings. Then, in a React app:

```
npm install @supabase/supabase-js
```

supabase.js

```
import { createClient } from "@supabase/supabase-js";

export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
);

// Register
await supabase.auth.signUp({ email, password });
// Login
await supabase.auth.signInWithPassword({ email, password });
// Logout
await supabase.auth.signOut();

// Read data
const { data, error } = await supabase.from("students").select("*");
// Insert
await supabase.from("students").insert({ name: "Ravi", email: "r@example.com" });
```

Row Level Security (RLS)

RLS decides which rows each user may read or change. Without correct RLS, anyone with your public key could access data. Turn RLS on for every table that holds user data and write policies.

Example policy

```
alter table students enable row level security;

create policy "Logged-in users can read students"
on students for select
to authenticated
using (true);
```

Storage

Create a bucket in the Storage section, decide if it is public or private, then upload from code with `supabase.storage.from("avatars").upload(path, file)`. Add policies for who may upload.

Practical project: Student Management App
Tables: courses, students. Pages: login, student list, add/edit form. Use RLS so only logged-in users can write. Deploy it in Part 10.

Common mistakes
Forgetting RLS; exposing secret keys; committing the .env file; mixing up anon key and service role key.

MODULE 14

Firebase Complete Guide

Google's app platform: database, auth, storage and hosting

Firebase Complete Guide

Firebase is Google's platform for building apps without managing servers. Official site: https://firebase.google.com and documentation at https://firebase.google.com/docs. It has a no-cost starting plan; confirm current limits and billing rules on the pricing page, especially for features that may need a billing account.

Setup

- Sign in to the Firebase console with a Google account.
- Add project, name it, and finish the wizard.
- Add a Web app and copy the configuration object.
- Enable products you need: Authentication, Firestore, Storage, Hosting.

Core services

Service	Purpose
Authentication	Email/password, Google sign-in and other providers.
Firestore	NoSQL database of collections and documents.
Storage	Store files such as images.
Hosting	Host static or single-page web apps.
Security Rules	Control who can read or write data.

React integration

```
npm install firebase
```

firebase.js

```
import { initializeApp } from "firebase/app";
import { getFirestore, collection, addDoc, getDocs } from "firebase/firestore";
import { getAuth, signInWithEmailAndPassword,
         createUserWithEmailAndPassword } from "firebase/auth";

const app = initializeApp({
  apiKey: import.meta.env.VITE_FB_API_KEY,
  authDomain: import.meta.env.VITE_FB_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FB_PROJECT_ID,
});
export const db = getFirestore(app);
export const auth = getAuth(app);

await addDoc(collection(db, "students"), { name: "Asha", course: "Web" });
const snap = await getDocs(collection(db, "students"));
snap.forEach(d => console.log(d.id, d.data()));
```

Security rules

Firestore rules (only signed-in users)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /students/{id} {
      allow read, write: if request.auth != null;
    }
  }
}
```

Never leave test mode open
Rules that allow everyone to read and write are for short experiments only. Anyone could read or delete your data.

Hosting

Deploy

```
npm install -g firebase-tools
firebase login
firebase init hosting
npm run build
firebase deploy
```

Supabase vs Firebase

	Supabase	Firebase
Database	PostgreSQL (SQL, relations)	Firestore (NoSQL documents)
Best for	Structured, relational data, SQL fans	Fast start, real-time apps, Google ecosystem
Auth	Built in	Built in
Security	Row Level Security policies	Security Rules
Open source	Core is open source	Proprietary Google service
Lock-in	Lower (standard Postgres)	Higher (Firebase-specific)

Practical app
Build a note-taking app: sign-up, login, add notes per user, rules so users only see their own notes. Use the same app idea in Supabase to feel the difference.

MODULE 15

Domain Name Registration Guide

Get your own address on the internet

Domain Name Registration Guide

A **domain name** is the human-friendly address of a website, such as skyrovix.com. It is rented, usually per year, from a **registrar** (a company authorised to sell domains). Official ICANN beginner information: https://www.icann.org

Domain extensions

Extension	Meaning / use
.com	Most common commercial extension
.in	India-focused
.org	Often organisations and communities
.online	Newer extension; often cheaper first year

Choosing a good name

- Short, easy to spell and say aloud.
- No hyphens or confusing numbers.
- Check that matching social media names are free.
- Avoid trademarks of others.

Registration steps (general)

- Search the name on the registrar site.
- Add to cart and check the **total including renewal price**. First-year offers often rise at renewal.
- Create an account, enter accurate contact details (you must be reachable).
- Pay and confirm the email verification.
- Turn on two-step login for your account.

GoDaddy and BigRock

Both are registrars with Indian users. Websites: https://www.godaddy.com and https://www.bigrock.in. Screens change, so follow the headings below rather than exact button positions.

Task	Where to look
Buy domain	Search bar on the home page, then Cart and Checkout
See your domains	Account menu, then My Products or Domain list
DNS records	Open the domain, then choose DNS / Manage DNS
Nameservers	Domain settings, then Nameservers
Renewal	Domain settings, auto-renew option

Prices in INR
Registrars show INR prices on their Indian sites, but they vary by extension, offer and tax, and change often. I have not printed prices. Check the final cart total and the renewal price before paying.

DNS explained

Record	Purpose	Example
A	Points a name to an IPv4 address	@ -> the IP your host gives you
CNAME	Points a name to another name	www -> hosting provider address
TXT	Verification or email policies	Used to prove domain ownership
MX	Email servers	Used with email services
Nameserver (NS)	Which servers control your DNS	Switch to your host's nameservers if told to

TTL is how long DNS answers are cached. Changes can take from minutes to many hours to spread (called **propagation**).

Connecting a domain to hosting

- Choose method: change nameservers to your host, **or** keep the registrar DNS and add the records your host lists.
- Copy the exact values from your hosting dashboard; do not guess.
- Delete conflicting old A or CNAME records for the same name.
- Wait, then test with https://www.whatsmydns.net or the host dashboard status.

Common errors

Problem	Likely cause
Site not found	DNS not propagated or wrong record
Shows registrar parked page	Old A record still present
Certificate error	Domain not yet verified by host
www works, root does not	Missing @ record or redirect

Practical
Pick a name, check availability on two registrars, compare total first-year and renewal costs in a table, and write down the DNS records you would need for your host (done for real in Part 10).

MODULE 16

Vercel Deployment Complete Guide

From your laptop to a live website

Vercel Deployment Complete Guide

Vercel is a platform that builds and hosts web projects, especially React and Next.js. Official site: https://vercel.com, docs: https://vercel.com/docs. It has a free Hobby plan for personal projects; verify current limits and terms on the pricing page. **Deployment** means publishing your project so everyone can open it.

Step 1: Prepare the project

Local computer

```
npm create vite@latest skyrovix-site -- --template react
cd skyrovix-site
npm install
npm run dev      # test locally
npm run build    # make sure the build works
```

Step 2: Push to GitHub

- Create a GitHub account and a new empty repository.
- Make sure .env files are listed in .gitignore.

```
git init
git add .
git commit -m "First version"
git branch -M main
git remote add origin https://github.com/YOUR-NAME/skyrovix-site.git
git push -u origin main
```

Step 3: Import into Vercel

- Sign up at vercel.com using GitHub.
- Choose Add New Project and select your repository.
- Vercel detects the framework. Confirm build command (npm run build) and output directory (for Vite: dist; for Next.js it is automatic).
- Add environment variables (below), then click Deploy.
- You get a vercel.app address. Open it to see the live site.

Environment variables

These hold secrets and settings outside your code, e.g. VITE_SUPABASE_URL. Add them in Project Settings > Environment Variables. Vite exposes only variables starting with VITE_ to the browser; Next.js uses NEXT_PUBLIC_ for the same purpose. Never put real secrets in those public variables.

React vs Next.js

	React (Vite)	Next.js
Type	Browser-rendered app	Framework with server features and routing
Vercel setup	Detected automatically	Detected automatically
Good for	Dashboards, learning	Public sites, SEO, full-stack

Custom domain

- Project Settings > Domains > add your domain (e.g. example.com).
- Vercel shows the DNS records required for your setup. Copy them exactly.
- In GoDaddy or BigRock open DNS management and add those records (typically an A record for the root and a CNAME for www), or change nameservers if Vercel instructs that.
- Wait for verification. Vercel provisions an **SSL certificate** (the padlock, HTTPS) automatically once DNS is correct.

Updating the site

Every push redeploys

```
git add .
git commit -m "Update contact page"
git push
```

Vercel builds each push; branches get **preview** URLs so you can test before merging to main (the production site).

Troubleshooting

Error	What to check
Build failed	Read the build log; run npm run build locally; check Node version and missing packages.
Blank page	Wrong output directory; console errors (F12); missing env variables.
404 on refresh	Single-page app routing; add a rewrite to index.html per Vercel docs.
Env variable undefined	Name prefix (VITE_/NEXT_PUBLIC_); redeploy after changing.
Domain not working	DNS records mismatch or still propagating.

Complete practical
Take your Part 01 capstone: 1) build locally, 2) push to GitHub, 3) import to Vercel, 4) add Supabase or Firebase variables, 5) deploy, 6) optionally connect a domain, 7) change one line and watch it redeploy.

Final conclusion

You now have the full path: learn the web, use AI tools carefully, set up your editor, design, build with a database, host and publish. Keep building real projects; they are your portfolio. Good luck from the SKYROVIX team.

MODULE 17

Advanced Full Stack Development

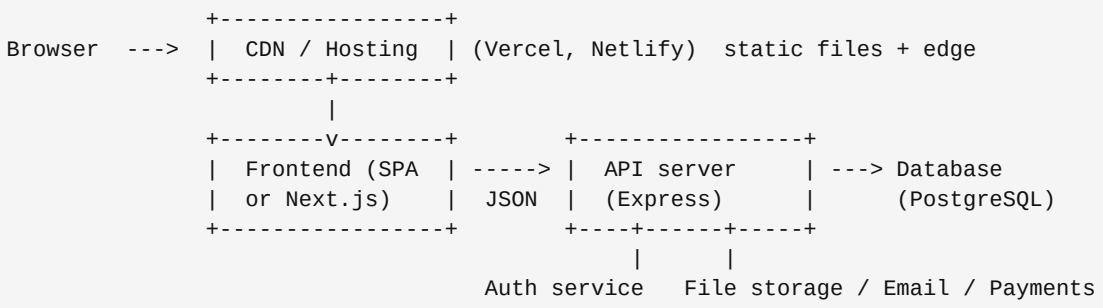
Architecture, auth systems, RBAC, payments, real-time, security, performance, SEO, testing, PWA

Advanced Full Stack Development

17.1 Full Stack Architecture

A full stack application has a **client** (React or Next.js), an **API** (Express or Next.js route handlers), a **database**, and supporting services (authentication, file storage, email, payments). Decide early between a **monolith** (one codebase, simple, ideal for most student and startup projects) and **microservices** (many small services, useful only at large scale because of operational cost).

Reference architecture



Choice	Beginner-friendly option	Why
Frontend	React + Vite or Next.js	Large community and docs
API	Express or Next.js route handlers	Same language everywhere
Database	PostgreSQL (Supabase)	Relational, free tiers, built-in auth and RLS
Hosting	Vercel + managed database	Git-based deploys

17.2 Frontend and Backend Integration

Keep the contract between frontend and backend explicit: documented endpoints, request and response shapes, and error format. Use environment variables for the API address so the same code works in development and production.

One API helper for the whole app

```
// src/api/client.js
const BASE = import.meta.env.VITE_API_URL;

export async function api(path, { method = "GET", body, token } = {}) {
  const res = await fetch(BASE + path, {
    method,
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    body: body ? JSON.stringify(body) : undefined,
  });
  const data = res.status === 204 ? null : await res.json().catch(() => null);
  if (!res.ok) throw Object.assign(new Error(data?.error || "Request failed"), { status: res.
status });
  return data;
}
// usage:  const students = await api("/students", { token });
```

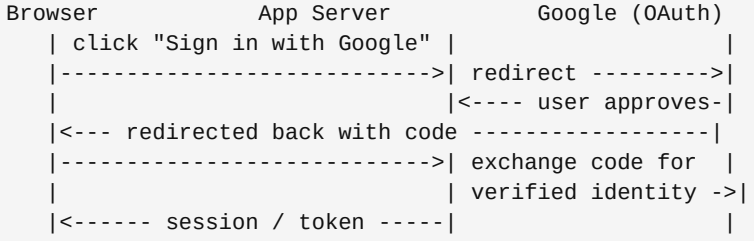
CORS in development: either allow your frontend origin on the server, or use the Vite dev proxy (`server.proxy` in vite.config) so the browser sees one origin. In production, set the allowed origin to your real domain only.

17.3 Authentication Systems

Authentication has more parts than login: registration, email verification, password reset, sessions or tokens, logout, and protection against abuse. Options: **build your own** (educational, risky in production), use an **auth service** (Supabase Auth, Firebase Auth, or others), or use **social login** (OAuth) with Google or GitHub.

Feature	Why it matters
Email verification	Stops fake or mistyped accounts
Password reset with expiring token	Users forget passwords; links must expire and be single-use
Rate limiting and lockout	Blocks password-guessing attacks
Short-lived access token + refresh	Limits damage if a token leaks
Multi-factor authentication	Protects against stolen passwords
Secure cookies (httpOnly, secure, sameSite)	Protects tokens from scripts and cross-site requests

OAuth flow in simple form



17.4 Role-Based Access Control

RBAC assigns permissions to **roles** and roles to users. Typical roles: admin, teacher, student. Always enforce on the server (or database policies), and mirror it in the UI only to hide controls.

Permission map

```
const PERMISSIONS = {
  admin:   ["students:read", "students:write", "students:delete", "courses:write"],
  teacher: ["students:read", "marks:write"],
  student: ["own:read"],
};
export const can = (role, perm) => PERMISSIONS[role]?.includes(perm) ?? false;

export const require = (perm) => (req, res, next) =>
  can(req.user?.role, perm) ? next() : res.status(403).json({ error: "Forbidden" });

app.delete("/api/students/:id", requireAuth, require("students:delete"), removeStudent);
```

RLS policy example (Supabase)

```
-- Same idea in PostgreSQL / Supabase with Row Level Security
create policy "Students read own marks"
on marks for select to authenticated
using (student_id = auth.uid());
```

17.5 Payment Gateway Integration

A **payment gateway** collects card, UPI or wallet payments securely. **Never handle raw card numbers yourself**; the provider's hosted checkout or SDK does that, which keeps you out of most compliance scope. Popular options for Indian developers include Razorpay (https://razorpay.com/docs/) and Stripe (https://docs.stripe.com). Read the provider's current documentation, supported payment methods and fees yourself before choosing, because they vary by country and change.

Safe payment flow

```
1. Frontend asks your server:  POST /api/orders  { cart }
2. Server calculates the amount FROM YOUR DATABASE (never trust the browser's price)
   and creates an order/session with the payment provider using the SECRET key.
3. Server returns the order id / checkout URL.
4. Browser opens the provider's checkout and the customer pays.
5. Provider calls YOUR webhook:  POST /api/webhooks/payment  (signed)
6. Server VERIFIES the signature, then marks the order as paid and sends the receipt.
7. Frontend shows success by reading the order status from your server.
```

- Use **test mode** keys while developing; switch to live keys only after review.
- Secret keys live only on the server in environment variables.
- Verify **webhook signatures**; the redirect back to your site alone is not proof of payment.
- Make webhooks **idempotent**: the same event may arrive twice.
- Store the provider payment id and status in your orders table.

17.6 E-commerce Development

An online store needs products, a cart, checkout, orders and an admin area.

Core tables

```
products(id, name, description, price_paise, stock, image_url, active)
customers(id, name, email, ...)
orders(id, customer_id, status, total_paise, payment_id, created_at)
order_items(order_id, product_id, quantity, unit_price_paise)
```

Store money as **integers in the smallest unit** (paise, cents) to avoid floating-point mistakes such as 0.1 + 0.2. Decrease stock inside a **transaction** when an order is created, and recalculate prices on the server.

Create an order safely

```
BEGIN;
UPDATE products SET stock = stock - 2 WHERE id = 7 AND stock >= 2;   -- fails if not enough
INSERT INTO orders (customer_id, status, total_paise) VALUES (3, 'pending', 199800) RETURNING
 id;
INSERT INTO order_items (order_id, product_id, quantity, unit_price_paise) VALUES (101, 7, 2,
 99900);
COMMIT;
```

Cart logic

```
// React cart with Context + reducer
function cartReducer(state, action) {
  switch (action.type) {
    case "add": {
      const found = state.find(i => i.id === action.item.id);
      return found
        ? state.map(i => i.id === found.id ? { ...i, qty: i.qty + 1 } : i)
        : [...state, { ...action.item, qty: 1 }];
    }
    case "remove": return state.filter(i => i.id !== action.id);
    case "clear": return [];
    default: return state;
  }
}
const total = cart.reduce((t, i) => t + i.price * i.qty, 0);   // display only
```

17.7 Admin Dashboard Development

An admin dashboard lets staff manage data. Typical pieces: protected routes, a sidebar layout, tables with search, sort and pagination, forms with validation, charts, and an audit trail of who changed what.

Paginated admin endpoint

```
// Server-side pagination and search
app.get("/api/admin/students", requireAuth, require("students:read"), async (req, res) => {
  const page = Math.max(Number(req.query.page) || 1, 1);
  const size = Math.min(Number(req.query.size) || 20, 100);
  const q = `%${req.query.q || ""}%`;
  const { rows } = await pool.query(
    "SELECT id, name, email FROM students WHERE name ILIKE $1 ORDER BY id LIMIT $2 OFFSET $3"
,
    [q, size, (page - 1) * size]);
  const total = (await pool.query("SELECT COUNT(*) FROM students WHERE name ILIKE $1", [q])).
rows[0].count;
  res.json({ rows, page, size, total: Number(total) });
});
```

Use a chart library such as Recharts or Chart.js for graphs (see their official docs). Log important admin actions (delete, role change) with the user id and time. The layout in Module 5 (Student Dashboard screenshot) is a good starting point.

17.8 Real-Time Applications

Normal HTTP is request and response. **Real-time** features (chat, live notifications, collaborative editing) need the server to push data. Options: **WebSockets** (two-way connection; library Socket.IO simplifies it), **Server-Sent Events** (one-way server to browser), **polling** (ask repeatedly; simplest), or managed realtime from Supabase Realtime or Firestore listeners.

Tiny chat with Socket.IO

```
// server (Socket.IO)
import { Server } from "socket.io";
const io = new Server(httpServer, { cors: { origin: process.env.CLIENT_URL } });
io.use((socket, next) => verifyToken(socket.handshake.auth.token) ? next() : next(new Error("
unauthorized")));
io.on("connection", (socket) => {
  socket.on("chat:send", (msg) => {
    if (typeof msg !== "string" || msg.length > 500) return;
    io.emit("chat:new", { text: msg, at: Date.now() });
  });
});

// client
import { io } from "socket.io-client";
const socket = io(import.meta.env.VITE_API_URL, { auth: { token } });
socket.on("chat:new", (m) => setMessages(prev => [...prev, m]));
socket.emit("chat:send", "Hello class");
```

Authenticate connections, validate every message, and rate-limit senders. Platforms like Vercel run serverless functions that do not keep long connections, so for WebSockets use a long-running server or a managed realtime service.

17.9 Web Security

Security is part of every feature. The **OWASP Top 10** (https://owasp.org/www-project-top-ten/) lists the most common web risks.

Risk	What it is	Defence
Injection	Malicious SQL or commands in input	Parameterised queries, validation
XSS	Script injected into a page	Escape output, avoid innerHTML, Content Security Policy
CSRF	Another site makes your browser send a request	sameSite cookies, CSRF tokens
Broken access control	Users reach data they should not	Server-side checks, RLS, test with other accounts
Weak authentication	Guessable passwords, no rate limits	Hashing, MFA, lockout
Sensitive data exposure	Secrets or personal data leaked	HTTPS, env vars, never log secrets

Risk	What it is	Defence
Vulnerable dependencies	Old packages with known flaws	npm audit, updates
Misconfiguration	Debug on, open ports, public buckets	Production checklist

Quick hardening for Express

```
import helmet from "helmet";
import rateLimit from "express-rate-limit";

app.use(helmet());
app.use("/api/auth", rateLimit({ windowMs: 15 * 60 * 1000, limit: 20 }));
app.disable("x-powered-by");
```

Rule of the three
Never trust (1) user input, (2) the browser, or (3) your own memory. Validate on the server, check permissions on the server, and test with a second account.

17.10 Performance Optimization

Speed affects users, SEO and conversion. Measure first with **Lighthouse** (Chrome DevTools) and **web.dev/measure**. Core Web Vitals track loading (LCP), responsiveness (INP) and layout stability (CLS); see https://web.dev/vitals/.

- **Images:** modern formats (WebP/AVIF), correct dimensions, lazy loading, responsive srcset.
- **JavaScript:** ship less; code-split routes with `React.lazy`; remove unused libraries.
- **Fonts:** limit weights, use `font-display: swap`.
- **Caching:** long cache for hashed assets; use a CDN (Vercel does this automatically).
- **Database:** add indexes, avoid fetching more than needed, paginate.
- **React:** avoid unnecessary re-renders; memoise only after measuring.

Route-level code splitting

```
import { lazy, Suspense } from "react";
const Dashboard = lazy(() => import("./pages/Dashboard"));

<Suspense fallback={<p>Loading...</p>}>
  <Routes><Route path="/dashboard" element={<Dashboard />} /></Routes>
</Suspense>
```

17.11 SEO

Search Engine Optimisation helps people find your site. For single-page apps, search engines may struggle with content that appears only after JavaScript runs, which is a main reason to choose Next.js (server rendering) for public sites. Checklist: unique titles and descriptions per page, one h1, semantic HTML, fast pages, mobile friendly, clean URLs, sitemap.xml, robots.txt, structured data, and HTTPS. Submit your sitemap in Google Search Console (https://search.google.com/search-console).

Metadata in Next.js

```
// Next.js App Router metadata
export const metadata = {
  title: "Web Development Course | SKYROVIX",
  description: "Learn HTML, CSS, JavaScript and React with real projects.",
  openGraph: { title: "Web Development Course", images: ["/og.png"] },
};
```

17.12 Web Application Testing

Tests protect you when you change code. Common levels: **unit** (one function), **integration** (several parts, such as an API route with the database), and **end-to-end** (a real browser clicking through the app). Tools: Vitest or Jest (unit), React Testing Library (components), Supertest (Express APIs), Playwright (browser). Docs: https://vitest.dev and https://playwright.dev.

A unit test

```
// utils/price.test.js (Vitest)
import { describe, it, expect } from "vitest";
import { formatINR, cartTotal } from "./price";

describe("price helpers", () => {
  it("formats paise as rupees", () => expect(formatINR(199800)).toContain("1,998"));
  it("sums the cart", () => expect(cartTotal([{ price: 100, qty: 2 }, { price: 50, qty: 1 }])
).toBe(250));
});
```

An end-to-end test

```
// e2e/login.spec.js (Playwright)
import { test, expect } from "@playwright/test";
test("user can log in", async ({ page }) => {
  await page.goto("http://localhost:5173/login");
  await page.getByLabel("Email").fill("asha@example.com");
  await page.getByLabel("Password").fill("correct-horse-battery");
  await page.getByRole("button", { name: "Log in" }).click();
  await expect(page.getByText("Welcome")).toBeVisible();
});
```

17.13 Progressive Web Apps

A **PWA** is a website that can be installed on a phone or desktop, work offline and send notifications. Requirements: HTTPS, a **web app manifest**, and a **service worker** (a script that intercepts network requests and can cache them). Details: https://web.dev/learn/pwa.

manifest.webmanifest

```
{
  "name": "SKYROVIX Learning",
  "short_name": "SKYROVIX",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#FFFFFF",
  "theme_color": "#F95C4B",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

Minimal service worker

```
// service-worker.js : cache the shell, serve cached copy if offline
const CACHE = "skyrovix-v1";
self.addEventListener("install", e =>
  e.waitUntil(caches.open(CACHE).then(c => c.addAll(["/", "/index.html", "/styles.css"]))));
self.addEventListener("fetch", e =>
  e.respondWith(caches.match(e.request).then(r => r || fetch(e.request))));
```

For Vite projects, the plugin **vite-plugin-pwa** generates these files for you. Test offline mode with DevTools, Application tab. Be careful with caching: stale files cause confusing bugs, so use versioned cache names.

17.14 Production Best Practices

Before launching, go through a checklist.

Area	Check
Config	Secrets only in environment variables; .env never committed; separate dev and production settings
Security	HTTPS, helmet, rate limits, CORS restricted, dependencies audited
Database	Backups tested, indexes in place, migrations in version control, least-privilege user
Errors	Friendly messages to users; detailed logs for you; error monitoring (e.g. Sentry)
Performance	Lighthouse score checked, images optimised, caching working
SEO	Titles, descriptions, sitemap, robots.txt
Legal	Privacy policy, cookie notice where required, terms
Process	Git branches, pull requests, CI running tests, staging environment, rollback plan

Module 17: Common Mistakes and Troubleshooting

- CORS errors after deploying: the allowed origin still points to localhost.
- Webhooks not arriving: wrong URL, missing public HTTPS endpoint, or signature verification failing because the body was parsed before verification (use the raw body).
- Prices tampered with in the browser: always calculate on the server.

- Auth works locally but fails in production: cookies need secure and the correct sameSite and domain settings.
- WebSockets fail on serverless hosting.
- Stale service worker caches: bump the cache name.

Module 17: Best Practices and Security

- Threat-model each feature: what could an attacker or a careless user do?
- Log security events and monitor errors.
- Automate tests and deployments (CI/CD with GitHub Actions).
- Keep payment, auth and personal data handling to proven services.
- Review OWASP checklists before launch.

Module 17: Exercises

Beginner exercises

- Add a role field to users and show different menus per role.
- Make a cart that persists in localStorage.
- Run Lighthouse on your project and write down three improvements.

Advanced exercises

- Implement RBAC on the server with a permission map and tests that prove a student cannot delete.
- Integrate a payment provider in test mode with a verified webhook.
- Add Socket.IO notifications to a dashboard.
- Turn your app into an installable PWA.

Mini project

Build a small shop: product list, cart, checkout in payment test mode, and an admin page to add products.

Module assignment

Take your Student Management System and add: roles (admin, teacher, student), a paginated admin dashboard, Playwright login test, security headers and rate limiting, a Lighthouse report with fixes, and a production checklist filled in.

Module 17: Interview Questions

Question	Short model answer
Monolith vs microservices?	One deployable codebase vs many small services; start monolith.
Why must prices be calculated on the server?	The browser can be modified by the user.
What is a webhook?	A request the provider sends to your server when an event happens.
What is CSRF?	Tricking a logged-in browser into sending an unwanted request; use sameSite cookies and tokens.
What is a service worker?	A script that intercepts network requests to enable offline use and caching.
What are Core Web Vitals?	Google metrics for loading, interactivity and visual stability.
What is RBAC?	Access control based on user roles.

Module 17: Official Documentation and Resources

Resource	Link
OWASP Top 10	https://owasp.org/www-project-top-ten/
web.dev Learn PWA	https://web.dev/learn/pwa
Vitest	https://vitest.dev
Playwright	https://playwright.dev
Razorpay docs	https://razorpay.com/docs/
Stripe docs	https://docs.stripe.com
Socket.IO	https://socket.io/docs/v4/

MODULE 18

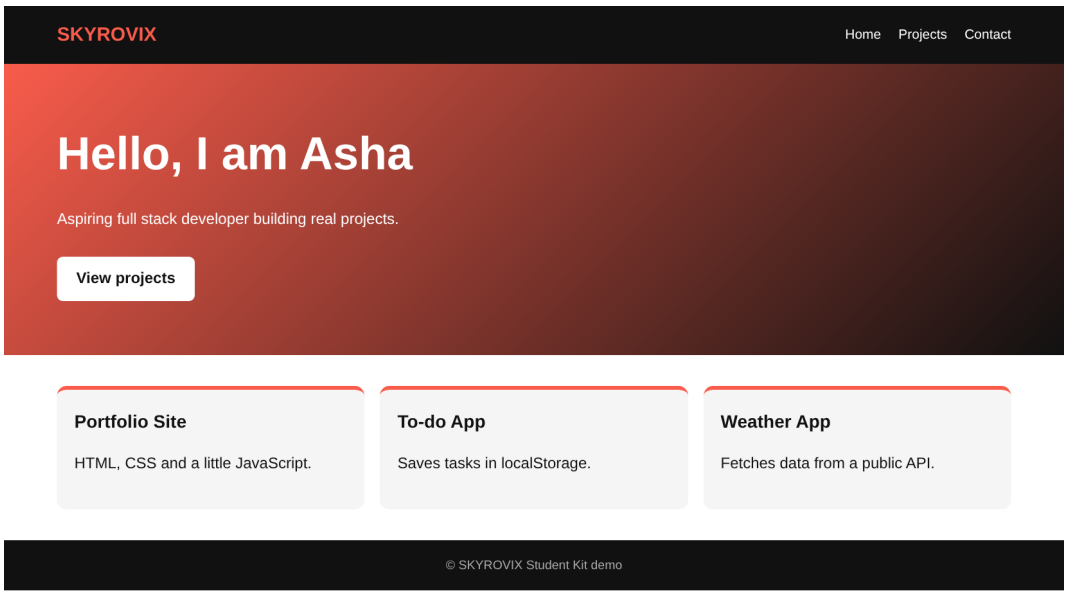
Practical Real-World Projects

Eleven guided projects from a portfolio site to a complete full stack application

Practical Real-World Projects

Learning sticks when you build. Each project below has a goal, features, a suggested stack, step-by-step milestones, acceptance checks and extensions. Do them in order; each reuses skills from earlier ones. For every project: use Git from the first minute, write a README, and deploy it (Module 16).

How to work: build the smallest working version first, then improve. When stuck for more than 30 minutes, write down what you expected, what happened, and what you tried; this alone often reveals the answer.



Screenshot: the kind of portfolio / landing page you will build in Projects 1 to 3 (Chromium render)

1. Personal Portfolio Website

Item	Details
Goal	Present yourself and your work to employers.
Features	Home (hero), About, Projects grid, Skills, Contact form, footer with links. Responsive, dark-mode toggle.
Suggested stack	HTML, CSS (Grid/Flexbox, variables), a little JavaScript.

Step-by-step milestones

1. Sketch the page on paper, then wireframe it.
2. Build semantic HTML with header, main sections and footer.
3. Style with CSS variables for the brand colours; make it mobile-first.
4. Add a smooth-scroll navbar and a projects grid with 3 real projects.
5. Add JavaScript for the mobile menu and dark mode saved in localStorage.
6. Deploy and add the link to your GitHub profile.

Acceptance checks

- Lighthouse accessibility 90+
- Works at 360px width
- All images have alt text

Extensions

- Add a blog section
- Add a downloadable resume PDF

2. Business Website

Item	Details
Goal	A 4-page site for a local business (e.g. a tuition centre or salon).
Features	Home, Services, Gallery, Contact with map link and enquiry form, WhatsApp link, SEO tags.
Suggested stack	HTML, CSS, JavaScript; a form service or your own backend for submissions.

Step-by-step milestones

1. Interview a real or imaginary client; list needs and content.
2. Create a sitemap and wireframes in Figma.
3. Build reusable header and footer markup.
4. Create services cards, an image gallery with lazy loading, and a validated contact form.
5. Add title and description meta tags, Open Graph tags, sitemap.xml and robots.txt.
6. Test on phone and deploy with a custom domain if possible.

Acceptance checks

- Valid HTML
- Form validates and shows success message
- Page loads fast with compressed images

Extensions

- Google Business and Search Console setup
- Basic analytics with a privacy notice

3. Landing Page

Item	Details
Goal	A single page designed to convert visitors into sign-ups.
Features	Hero with call to action, benefits, testimonials, pricing cards, FAQ accordion, sign-up form, footer.
Suggested stack	HTML, CSS, JavaScript, optionally Tailwind.

Step-by-step milestones

1. Write the copy first: headline, 3 benefits, a clear call to action.

2. Build the sections with Grid and Flexbox.
3. Create an accessible FAQ using details and summary.
4. Add form validation and a thank-you state.
5. Test contrast and keyboard use.

Acceptance checks

- One clear primary action
- Looks good at 3 breakpoints
- Form is keyboard accessible

Extensions

- A/B test two headlines
- Add scroll animations respecting reduced motion

4. Task Management Application

Item	Details
Goal	Create and track tasks with categories and due dates.
Features	Add, edit, delete, complete tasks; filter by status; search; due-date highlight; persist data.
Suggested stack	React, useState/useReducer, localStorage (then an API).

Step-by-step milestones

1. Define the task object: id, title, status, due date, priority.
2. Build components: TaskForm, TaskList, TaskItem, Filters.
3. Implement a reducer for add, update, delete and toggle.
4. Persist with a custom useLocalStorage hook.
5. Move storage to your Express + database API.
6. Add login so each user sees only their tasks.

Acceptance checks

- Refresh keeps tasks
- Empty and error states shown
- Another user cannot see your tasks

Extensions

- Drag and drop ordering
- Email reminders

5. Blog Application

Item	Details
Goal	Publish and read articles.
Features	Post list with pagination, post page, categories, markdown content, admin writing form, comments (optional).
Suggested stack	Next.js or React + Express, PostgreSQL.

Step-by-step milestones

1. Design tables: users, posts, categories, comments.
2. Create API routes for posts with pagination.
3. Build list and detail pages with good titles and meta tags.
4. Add an admin form to write posts, with validation.
5. Render markdown safely (sanitise output).
6. Add slugs for friendly URLs and a sitemap.

Acceptance checks

- Public pages readable without login
- Only admin can create posts
- Each post has unique meta tags

Extensions

- Search
- RSS feed
- Image upload for covers

6. Student Management System

Item	Details
Goal	Manage students, courses and marks (the capstone in Module 20 expands this).
Features	Login with roles, student CRUD with search and pagination, courses, enrollments, marks, profile page.
Suggested stack	React, Express, PostgreSQL (Supabase).

Step-by-step milestones

1. Draw the ER diagram.
2. Create tables with constraints.
3. Build auth and role middleware.
4. Build CRUD endpoints and test with Postman.
5. Build the React pages with protected routes.
6. Add pagination, validation and error handling.

Acceptance checks

- Admin only can delete
- Duplicate email rejected
- Mobile layout works

Extensions

- CSV import and export
- Attendance tracking

7. E-commerce Website

Item	Details
Goal	A small online shop.
Features	Product list and detail, cart, checkout (payment test mode), orders, admin product management.
Suggested stack	Next.js or React + Express, PostgreSQL, payment provider test mode.

Step-by-step milestones

1. Design product, order and order_item tables; store prices in paise.
2. Build the catalogue with filters and search.
3. Implement the cart with context and localStorage.
4. Create orders on the server and recalculate totals there.
5. Integrate the provider in test mode with a verified webhook.
6. Build an admin page for products and orders.

Acceptance checks

- Price cannot be changed from the browser
- Stock decreases once per paid order
- Order status updates after webhook

Extensions

- Discount codes
- Order emails
- Product reviews

8. Admin Dashboard

Item	Details
Goal	Insights and management in one place.
Features	Sidebar layout, summary cards, charts, data tables with sorting and filters, role-based menus.
Suggested stack	React, a chart library, Express API.

Step-by-step milestones

1. Sketch widgets and decide the data each needs.
2. Write SQL aggregate queries for the numbers.
3. Create endpoints that return summaries.
4. Build reusable StatCard, DataTable and Chart components.
5. Add loading skeletons and empty states.

Acceptance checks

- Numbers match the database
- Tables paginate on the server
- Menu hides pages the role cannot use

Extensions

- Date range filters
- Export to CSV

9. Course Management Platform

Item	Details
Goal	A mini learning platform.
Features	Courses with lessons, enrolment, progress tracking, teacher area to add lessons, student area to continue learning, quizzes.
Suggested stack	Next.js or React, Supabase (auth, database, storage).

Step-by-step milestones

1. Model courses, lessons, enrollments, progress, quizzes.
2. Use Supabase Auth and Row Level Security so students see only enrolled courses.
3. Build the course catalogue and a lesson viewer.
4. Track completed lessons and show a progress bar.
5. Add teacher forms for lessons and file uploads.
6. Add a quiz with scoring.

Acceptance checks

- RLS blocks other users data
- Progress persists
- Teacher cannot edit others courses

Extensions

- Certificates as PDF
- Discussion threads

10. AI-Powered Web Application

Item	Details
Goal	Add AI features to a real app: for example a study assistant that summarises notes or generates quiz questions.
Features	Text input, AI response, history, rate limiting, error handling, cost control.
Suggested stack	Next.js or Express backend, an AI provider API called only from the server.

Step-by-step milestones

1. Choose a narrow, useful task.
2. Create a backend route that calls the provider with a carefully written system prompt.
3. Keep the API key on the server only; never in frontend code.
4. Validate and limit input length; add per-user rate limits.
5. Handle slow responses with a loading state and timeouts.
6. Save results in the database and show history.
7. Add a clear note that AI output may be wrong.

Acceptance checks

- API key not visible in browser tools
- Handles provider errors gracefully
- Usage per user is limited

Extensions

- Streaming responses
- Feedback buttons to rate answers

11. Complete Full Stack Application

Item	Details
Goal	Combine everything into one polished product of your choice (for example the Module 20 capstone).
Features	Authentication, roles, CRUD, search, file upload, responsive UI, tests, deployment on a custom domain.
Suggested stack	Your best stack from Modules 5 to 17.

Step-by-step milestones

1. Write requirements and acceptance criteria.
2. Design UI in Figma and the database as an ER diagram.
3. Build backend, then frontend, committing daily.
4. Add tests for the most important flows.
5. Security review with the OWASP-based checklist.
6. Deploy, document, record a demo.

Acceptance checks

- All acceptance criteria pass on the live site
- README lets a stranger run it
- Zero secrets in the repo

Extensions

- CI with GitHub Actions
- Error monitoring

Module 18: Common Mistakes and Troubleshooting

- Starting too big: finish a small version before adding features.
- Copying a tutorial without changing anything. Always add one feature of your own.
- Skipping error and empty states.
- Not testing on a phone.
- Leaving secrets in the repository.

Module 18: Best Practices and Security

- Plan, build small, test, deploy, repeat.
- Write the README as you go.
- Keep a changelog of what you add.
- Ask a friend to try the app without help and watch where they get stuck.

Module 18: Exercises

Beginner exercises

- Complete Project 1 and publish it.
- Complete Project 3 using a different layout technique than in Project 1.
- Complete Project 4 with localStorage.

Advanced exercises

- Complete Project 6 with roles and RLS.
- Complete Project 8 with real aggregate queries.
- Complete Project 10 with safe server-side AI calls and rate limiting.

Mini project

Choose any two projects, finish them completely (live link, README, screenshots) and add them to your GitHub profile.

Module assignment

Submit a portfolio containing at least five projects from this module with live links, repositories and a short reflection for each: what was hard, what you learned, what you would change.

Module 18: Interview Questions

Question	Short model answer
How do you decide project scope?	List must-have features, build those first, and keep extras for later.
How do you prove a project works?	Acceptance criteria, tests, and a live demo.
Why deploy your projects?	Employers can use them; deployment reveals real-world problems.
How do you keep API keys safe?	Server only, environment variables, never committed.
What did you learn from your hardest project?	Explain the problem, your process, and the outcome.

Module 18: Official Documentation and Resources

Resource	Link
MDN Learn Web Development	https://developer.mozilla.org/en-US/docs/Learn_web_development
freeCodeCamp	https://www.freecodecamp.org
The Odin Project	https://www.theodinproject.com
roadmap.sh projects	https://roadmap.sh/projects

MODULE 19

Professional Developer Skills

GitHub portfolio, documentation, teamwork, freelancing and careers

Professional Developer Skills

19.1 GitHub Portfolio Development

Employers and clients often look at your GitHub profile before anything else. Build it deliberately.

- **Profile README:** create a repository with the same name as your username and add a README.md: who you are, skills, 3 featured projects, contact link.
- **Pin 4 to 6 best repositories.** Quality beats quantity.
- **Every project has a README:** what it does, screenshot or GIF, tech stack, how to run it, live link, what you learned.
- **Commit regularly with clear messages** ("Add login validation", not "update").
- **Never commit secrets.** If you do, rotate the key immediately; deleting the commit is not enough.
- Add a license for open projects (https://choosealicense.com).

README template

```
# Project Name
One-sentence description.

![Screenshot](docs/screenshot.png)

## Features
- Login and registration
- Student CRUD with search

## Tech stack
React, Vite, Express, PostgreSQL

## Run locally
1. git clone https://github.com/you/project.git
2. cp .env.example .env   (fill in values)
3. npm install && npm run dev

## Live demo
https://your-project.vercel.app
```

19.2 Technical Documentation

Good documentation saves hours. Types: README (start here), API reference (endpoints, parameters, examples, errors), architecture notes (why decisions were made), and comments (explain *why*, not what).

API documentation style

```
### POST /api/students
Creates a student. Requires role: admin.

**Body**
| field | type | required | notes |
|-------|------|----------|-------|
| name  | string | yes | 2-100 chars |
| email | string | yes | must be unique |

**Responses**
- 201 Created - returns the student
- 400 Bad Request - invalid input
- 401 / 403 - not logged in / not allowed
- 409 Conflict - email already exists
```

Write for a reader who has never seen the project. Test your own instructions on a clean machine.

19.3 Code Review

A **code review** is when a teammate reads your changes before they are merged (a **pull request**). It catches bugs, spreads knowledge and keeps style consistent.

As author	As reviewer
Keep pull requests small and focused	Be kind and specific; critique the code, not the person
Describe what and why; add screenshots	Ask questions instead of giving orders where unsure
Run tests and the linter first	Check logic, security, edge cases, readability
Respond to every comment	Praise good solutions; approve when ready

19.4 Debugging Techniques

Debugging is a skill with a method, not luck.

- **Reproduce** the bug reliably; write down the steps.
- **Read the error message** completely. Note the file and line number.
- **Isolate:** remove code or inputs until the bug disappears, then add back.
- **Inspect:** console.log, the debugger with breakpoints, the Network tab, database queries.
- **Form a hypothesis** and test one change at a time.
- **Binary search** with Git: `git bisect` finds the commit that introduced a bug.
- **Rubber duck:** explain the problem aloud, line by line.
- **Write a test** that fails, fix, and keep the test.

19.5 Team Collaboration

Most professional work is teamwork. Use **Git branches** and pull requests, never everyone pushing to main.

Branch workflow

```
git checkout -b feature/student-search     # create a branch
# ... work, commit ...
git push -u origin feature/student-search   # then open a Pull Request on GitHub
git pull origin main                        # update your branch regularly
git rebase main    # or merge main into your branch; resolve conflicts carefully
```

Agree on conventions: branch names, commit style, code formatting (Prettier, ESLint), and who reviews. Communicate early when blocked and write decisions down.

19.6 Project Management

Break big work into small tasks. Common approach: **Agile** with short cycles (sprints) of one to two weeks and a **Kanban board** (To do, In progress, Review, Done). GitHub Projects (https://docs.github.com/en/issues/planning-and-tracking-with-projects) is free for personal use and works well with issues. A good task is small (one day), clear, and has **acceptance criteria** such as "A logged-in teacher can add marks; students cannot".

Step	Output
Plan	Requirements list, user stories
Design	Wireframes, database design
Build	Small tasks in order of value
Test	Checklists and automated tests
Release	Deployment and notes
Review	What went well, what to change

19.7 Freelancing Fundamentals

Freelancing means working for clients as an independent. Start with small projects (landing pages, portfolios for local businesses), build testimonials, then raise prices. Check each platform's current rules and fees yourself.

- **Define the scope in writing:** pages, features, number of revisions, deadlines, what is not included.
- **Agree payment terms before starting:** a common approach is a deposit, then milestones.
- **Use a simple contract or agreement** covering ownership, payment and changes. For anything large, take legal advice; this book is not legal advice.
- **Price by value and time**, not by guesswork; track your hours at first.
- **Keep records** for taxes according to your country's rules.
- **Protect yourself:** never hand over final files or hosting access before agreed payment.

19.8 Client Communication

Clear communication wins repeat work. Reply promptly, confirm requirements in writing, give regular progress updates, and explain technical points in plain language. Say no politely to scope creep and offer a paid change instead.

Weekly update email template

```
Subject: Update on your website - Friday

Hi Priya,

Progress this week:
- Home and About pages are ready for your review: (link)
- Contact form is built; waiting for your email address for the receiver.

Next steps:
1. Please send feedback on the home page by Tuesday.
2. I will then build the Courses page.

Thanks,
Asha
```

19.9 Resume Preparation

One page for a fresher. Use a clean layout, consistent dates, and measurable achievements. Tailor it to each job.

Section	What to include
Header	Name, email, phone, city, GitHub, LinkedIn, portfolio link
Summary (2 lines)	Role you target and your strongest skills
Skills	Grouped: Languages, Frontend, Backend, Databases, Tools
Projects (most important)	Name, 1-line purpose, tech, 2-3 bullets with results, GitHub and live links
Experience / internship	Role, dates, what you built or improved
Education and certificates	Degree, year, relevant courses

Good bullet: *Built a student management app with React and PostgreSQL; added role-based access and search, reducing lookup time from minutes to seconds in demo.* Weak bullet: *Worked on a website.* Only list skills you can discuss in an interview.

19.10 LinkedIn Profile Development

Use a clear photo, a headline that states your target ("Full Stack Developer | React, Node.js, PostgreSQL"), an About section telling your story, featured projects, and skills. Share what you learn, connect with developers and recruiters, and follow companies you want to work for. Keep every claim truthful and consistent with your resume.

19.11 Technical Interview Preparation

Interviews usually include: HTML/CSS/JavaScript questions, a coding problem, a project discussion, and basic behavioural questions.

- **Know your projects deeply:** why you chose each tool, one bug you solved, what you would improve.
- **Practise problems** on arrays, strings, objects, recursion and basic data structures.

- **Think aloud** and ask clarifying questions before coding.
- **Review fundamentals:** closures, the event loop, CSS layout, HTTP, REST, SQL joins, authentication.
- **Prepare questions** to ask the interviewer.

Practice example

```
// Typical warm-up: count how many times each word appears
function wordCount(text) {
  const counts = {};
  for (const w of text.toLowerCase().match(/[a-z']+/g) ?? []) {
    counts[w] = (counts[w] || 0) + 1;
  }
  return counts;
}
console.log(wordCount("To be or not to be"));   // { to: 2, be: 2, or: 1, not: 1 }
```

19.12 Career Roadmap

A realistic path. Times depend on your effort and background.

Stage	Focus	Goal
Months 0-3	HTML, CSS, JavaScript, Git	3 solid static/JS projects on GitHub
Months 3-6	React, APIs, responsive design	Deployed React app with live link
Months 6-9	Node, SQL, auth, deployment	Full stack project with login
Months 9-12	Testing, TypeScript, Next.js, DevOps basics	Capstone, resume, internship applications
Year 2+	Specialise (frontend, backend, AI apps)	Junior role, freelancing or product work

Module 19: Common Mistakes and Troubleshooting

- Applying with no projects live: always deploy.
- Resume lists skills you cannot explain.
- Tutorial hell: copying along without building your own variations.
- Committing secrets or huge files to Git.
- Not asking for help after being stuck for hours.

Module 19: Best Practices and Security

- Protect your accounts with two-factor authentication.
- Ask permission before using client or employer code publicly.
- Respect licenses of libraries you use.
- Be honest about experience and AI use.
- Keep learning in small daily steps.

Module 19: Exercises

Beginner exercises

- Create your GitHub profile README.
- Write a README for your best project.
- Create a one-page resume.

Advanced exercises

- Make a pull request to an open-source project's documentation.
- Record a 2-minute video walkthrough of a project.
- Do a mock interview with a friend and review it.

Mini project

Polish three projects: READMEs with screenshots, live links, and a short write-up of challenges and solutions.

Module assignment

Build your professional pack: GitHub profile README, three polished repositories, one-page resume, LinkedIn profile text, and a 60-second self-introduction script.

Module 19: Interview Questions

Question	Short model answer
Tell me about yourself.	Short, structured: background, skills, a project you are proud of, what you want next.
Describe a bug you fixed.	State symptoms, how you investigated, the cause, the fix and what you learned.
What is a pull request?	A proposal to merge a branch, reviewed before merging.
How do you handle disagreement in a team?	Listen, use data, discuss trade-offs, agree and document.
How do you estimate tasks?	Break into small pieces, compare with past work, add buffer, update as you learn.
How do you keep learning?	Projects, documentation, small experiments, reviewing others' code.

Module 19: Official Documentation and Resources

Resource	Link
GitHub Docs	https://docs.github.com
Pro Git book	https://git-scm.com/book
Choose a license	https://choosealicense.com
web.dev	https://web.dev
roadmap.sh	https://roadmap.sh

MODULE 20

Final Capstone Project

Plan, design, build, secure, test and launch a complete full stack application

Final Capstone Project: SKYROVIX Student Management System

This capstone ties every module together. You will build, document and deploy a **Student Management System** (SMS) and present it as a portfolio piece. Work in small steps, commit often, and write down decisions as you go.

20.1 Project Planning

Define the goal in one sentence: *Staff can manage students, courses and enrollments; students can log in and see their own courses and marks.* Then set the scope: what is included in version 1 and what waits for later.

Phase	Duration (suggested)	Output
Plan and requirements	2 days	User stories, scope, schedule
Design (UI + database)	4 days	Wireframes, ER diagram
Backend and database	7 days	API with auth and CRUD
Frontend	7 days	Responsive React app
Integration and security	3 days	Working end to end
Testing and fixes	3 days	Test report
Deployment and documentation	3 days	Live URL, README, demo video

20.2 Requirement Analysis

Write requirements as **user stories**: "As a [role], I want [feature], so that [benefit]". Add **acceptance criteria**.

ID	User story	Acceptance criteria
US1	As an admin I can add a student	Name and unique email required; student appears in list
US2	As an admin I can enrol a student in a course	Cannot enrol twice; shown on student page
US3	As a teacher I can record marks	Marks 0-100; only for my courses
US4	As a student I can view my courses and marks	See only my own data
US5	As any user I can log in and out	Wrong password gives a generic error

- **Functional:** login, roles, CRUD, search, pagination.
- **Non-functional:** responsive layout, page load under 3 seconds on a normal connection, passwords hashed, HTTPS, accessible forms.
- **Out of scope for v1:** payments, email notifications, real-time chat.

20.3 UI/UX Design

Design in Figma (Module 12) before coding: login, dashboard, student list, student form, course list, marks page, profile. Define colours (#F95C4B primary, #111111 text, #F5F5F5 background), a type scale and components (button, input, table, card, modal). Include empty states, loading states and error states. The layout in Module 5 (Student Dashboard screenshot) shows the direction.

20.4 Frontend Development

Suggested stack: React with Vite, React Router, a simple CSS or Tailwind setup, and the API helper from Module 17.

Frontend structure

```
src/
  api/client.js
  context/AuthContext.jsx
  components/  Navbar.jsx  Sidebar.jsx  DataTable.jsx  StudentForm.jsx  ProtectedRoute.jsx
  pages/       Login.jsx  Dashboard.jsx  Students.jsx  StudentDetail.jsx  Courses.jsx  Marks.
jsx
  hooks/       useFetch.js
  App.jsx  main.jsx
```

ProtectedRoute.jsx

```jsx
export default function ProtectedRoute({ roles, children }) {
  const { user, loading } = useAuth();
  if (loading) return <p>Loading...</p>;
  if (!user) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.role)) return <h2>Not allowed</h2>;
  return children;
}
// <Route path="/students" element={<ProtectedRoute roles={["admin","teacher"]}><Students/></
ProtectedRoute>} />
```

20.5 Backend Development

Express API with routes grouped by resource, middleware for auth and validation, and a central error handler.

Backend structure

```
server/
  server.js
  db.js                pool setup
  middleware/  auth.js  roles.js  validate.js  errors.js
  routes/      auth.js  students.js  courses.js  enrollments.js  marks.js
  .env.example
```

Endpoint	Roles	Purpose
POST /api/auth/login	public	Get token
GET /api/students	admin, teacher	List with search and pagination
POST /api/students	admin	Create student
PUT /api/students/:id	admin	Update

Endpoint	Roles	Purpose
DELETE /api/students/:id	admin	Remove
POST /api/enrollments	admin	Enrol student in course
PUT /api/marks	teacher	Record marks
GET /api/me/courses	student	Own courses and marks

20.6 Database Integration

Use PostgreSQL (for example via Supabase, Module 13).

schema.sql

```
CREATE TABLE users (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email TEXT UNIQUE NOT NULL, password_hash TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('admin','teacher','student')));
CREATE TABLE students (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id BIGINT UNIQUE REFERENCES users(id) ON DELETE SET NULL,
  name TEXT NOT NULL, phone TEXT, joined_on DATE DEFAULT CURRENT_DATE);
CREATE TABLE courses (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title TEXT NOT NULL, teacher_id BIGINT REFERENCES users(id));
CREATE TABLE enrollments (
  student_id BIGINT REFERENCES students(id) ON DELETE CASCADE,
  course_id  BIGINT REFERENCES courses(id)  ON DELETE CASCADE,
  marks INTEGER CHECK (marks BETWEEN 0 AND 100),
  PRIMARY KEY (student_id, course_id));
CREATE INDEX idx_enrollments_course ON enrollments(course_id);
```

20.7 Authentication and Security

Implement bcrypt hashing, JWT with short expiry, role middleware, validation on every write route, rate limiting on login, helmet, restricted CORS, parameterised queries, and tests proving that a student cannot read another student's marks. Walk through the Module 17 production checklist.

20.8 API Integration

Connect each page to its endpoint through the single API helper. Handle loading, empty, error and success states on every screen. Show server validation messages next to form fields. Add a toast or alert for successful actions.

20.9 Testing and Debugging

Create a test plan table and tick each row before release.

Test	Expected result	Pass?
Login with wrong password	Generic error, no token	
Student opens /students	Redirected or "Not allowed"	

Test	Expected result	Pass?
Student requests another student's marks via API	403	
Create student with duplicate email	409 and message	
Enrol the same student twice	Rejected	
Phone width (390px)	No horizontal scroll, readable	
Keyboard only	Can log in and add a student	
Lighthouse	Performance and accessibility reasonable; fix issues	
Refresh on /students after deploy	Page loads (SPA rewrite works)	

20.10 GitHub Repository Management

Use a clean history: `main` always works; develop features on branches; open pull requests and review them yourself; tag a release (`v1.0.0`). Include .gitignore (node_modules, .env), .env.example, README with screenshots, LICENSE, and a docs folder with the ER diagram and API table. Optionally add GitHub Actions to run tests on every push (https://docs.github.com/en/actions).

20.11 Domain Configuration

If you own a domain (Module 15), add it in Vercel (Module 16), copy the DNS records Vercel shows, add them in your registrar's DNS panel, wait for propagation and confirm that HTTPS works. Update your server's allowed origin (CORS) and any OAuth redirect URLs to the new domain.

20.12 Production Deployment

Deploy the frontend to Vercel and the API to a Node host of your choice (check each provider's current free options and limits yourself). Set all environment variables, run database migrations on the production database, create the first admin user safely, and test the full flow on the live URL. Take screenshots of the real running system for your documentation.

20.13 Final Project Documentation

Deliver: (1) GitHub repository link, (2) live URL, (3) README, (4) architecture diagram, (5) ER diagram, (6) API table, (7) test results, (8) a 3-minute demo video, (9) a short reflection on what you learned and what you would improve.

Module 20: Common Mistakes and Troubleshooting

- Scope creep: stop adding features once version 1 criteria are met.
- Environment differences: works locally but not live; compare environment variables and Node version.
- Forgetting database migrations on production.
- Leaving test accounts and weak passwords in production.
- Not documenting setup steps.

Module 20: Best Practices and Security

- Commit small and often; deploy early and repeatedly rather than once at the end.
- Test with real-looking data and with two different roles.
- Back up the database before risky changes.
- Keep the README honest about limitations.

Module 20: Exercises

Beginner exercises

- Write 10 user stories for the SMS.
- Create the ER diagram and the Figma wireframes.
- Build login and one protected page.

Advanced exercises

- Add CSV export of students for admins.
- Add an audit log table recording who changed what.
- Add a GitHub Actions workflow that lints and tests.

Mini project

Complete version 1 of the SMS with all user stories US1 to US5 working locally.

Module assignment

Submit the final capstone: live URL, repository, documentation package and demo video, plus a one-page reflection.

Module 20: Interview Questions

Question	Short model answer
Walk me through your architecture.	Describe client, API, database, auth flow and hosting, and why each was chosen.
How did you secure it?	Hashing, JWT, role checks on the server, validation, parameterised queries, rate limits, HTTPS.
What was the hardest bug?	Explain symptom, investigation, root cause, fix.
How would it scale?	Indexes, pagination, caching, separate services if needed, monitoring.
What would you improve?	Tests, notifications, better UX, CI/CD.

Module 20: Official Documentation and Resources

Resource	Link
Vercel docs	https://vercel.com/docs
Supabase docs	https://supabase.com/docs
GitHub Actions	https://docs.github.com/en/actions
OWASP Cheat Sheets	https://cheatsheetseries.owasp.org

Glossary

Term	Meaning
API	A defined way for programs to talk to each other.
Async	Code that waits for something without freezing the page.
Backend	Code and data running on a server.
Bundler	A tool (e.g. Vite) that prepares code for browsers.
Component	A reusable piece of user interface.
CORS	Browser rule limiting requests between different websites.
CSS	Language that styles web pages.
DNS	System that converts domain names into IP addresses.
DOM	The browser's tree of page elements.
Domain	A human-readable website address.
Frontend	What users see and use in the browser.
Git	Tool that records the history of code.
HTML	Markup language that structures web pages.
HTTP/HTTPS	Rules for sending web data; HTTPS is encrypted.
JSON	A text format for data using keys and values.
npm	Package manager that installs JavaScript libraries.
Repository	A project folder tracked by Git.
REST	Style of API using URLs and HTTP methods.
RLS	Row Level Security: database rules for who sees which rows.
SSL/TLS	Encryption behind HTTPS and the padlock.
SQL	Language for working with relational databases.
XSS	Attack that injects scripts into a page.

Conclusion and Next Steps

You have worked through the full path: how the web works, HTML, CSS, JavaScript, React and Next.js, Node and Express, databases, AI tools, development tools, design, Supabase and Firebase, domains, deployment, advanced full stack topics, real projects, professional skills and a capstone. The next step is the most important one: build, deploy and share real projects.

Honest limitations of this edition: screenshots are real browser renders of the book's own example code; screenshots of third-party software interfaces (VS Code, Figma, Vercel, etc.) are not included because they could not be captured, so follow each tool's current official documentation. Modules 8 to

16 are overview-level. Prices are intentionally not printed; check each official pricing page.